

Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών
Υπολογιστών

6ο Εξάμηνο: Βάσεις Δεδομένων

Εξαμηνιαία Εργασία: “HospitalDB”

Περίληψη

Η παρούσα αναφορά περιγράφει την πορεία σχεδιασμού και υλοποίησης της βάσης δεδομένων HospitalDB. Το έργο ξεκίνησε από το διάγραμμα Οντοτήτων-Συσχετίσεων, μετατράπηκε σε σχεσιακό σχήμα και στη συνέχεια υλοποιήθηκε σε MySQL με πίνακες, πρωτεύοντα και ξένα κλειδιά, περιορισμούς CHECK, triggers, stored procedures και views. Ιδιαίτερη έμφαση δόθηκε στους κανόνες που δεν εκφράζονται επαρκώς ως απλοί περιορισμοί πίνακα, όπως οι κανόνες εφημεριών, η ιεραρχία γιατρών, η αποφυγή επικάλυψης χειρουργείων, η ασφάλεια συνταγογράφησης και η FIFO διαχείριση της ουράς επειγόντων.

Ζώντος Ευστάθιος (el23019)

Κατσέτης Διονύσιος (el23005)

Λουκρέζης Αλέξανδρος (el23039)

Περίληψη.....	1
A. Σχεδιασμός και υλοποίηση Βάσης Δεδομένων.....	3
A.1. Διάγραμμα Οντοτήτων – Συσχετίσεων (Entity-Relationship Diagram).....	3
A.1.1 Ανάλυση Οντοτήτων (Entities).....	3
A.1.2 Σχόλια για την μοντελοποίηση των οντοτήτων.....	4
A.1.3 Ανάλυση Συσχετίσεων (Relationships).....	5
A.2. Σχεσιακό Διάγραμμα και υλοποίηση Βάσης Δεδομένων (Relational Model).....	9
A.2.1. Μετάβαση από το ER στο Relational.....	9
A.2.2. Derived Attributes.....	12
A.2.3. Απαραίτητοι Περιορισμοί για ορθότητα.....	13
A.2.3.1 Table Level Constraints.....	14
A.2.3.2 Triggers.....	19
A.2.3.3 Στρατηγική σε DELETE - UPDATE.....	24
A.2.4. Stored Procedures.....	25
A.2.5. Ορισμός Ευρετηρίων (Indexing).....	30
A.2.6. Views.....	32
A.4 Load και Synthetic Data.....	34
B. Queries.....	35
B.1.....	35
B.2.....	36
B.3.....	37
B.4.....	38
A. EXPLAIN / EXPLAIN ANALYZE.....	38
Αποτελέσματα EXPLAIN.....	38
Αποτελέσματα EXPLAIN ANALYZE.....	39
B. FORCE INDEX.....	40
C. ESTIMATED COST VS ACTUAL EXECUTION TIME.....	40
D. ANALYSIS OF ESTIMATED COST & ACTUAL TIME.....	40
B.5.....	41
B.6.....	41
A. EXPLAIN / EXPLAIN ANALYZE.....	43
Αποτελέσματα EXPLAIN.....	43
Αποτελέσματα EXPLAIN ANALYZE.....	44
B. Αποτελέσματα FORCE INDEX.....	44
C. ESTIMATED COST VS ACTUAL EXECUTION TIME.....	45
D. ANALYSIS OF ESTIMATED COST & ACTUAL TIME.....	45
B.7.....	45
B.8.....	46
B.9.....	47
B.10.....	48
B.11.....	49
B.12.....	49

B.13.....	51
B.14.....	52
B.15.....	52
Γ. Front End.....	54
1. Επίπεδο Πρόσβασης Δεδομένων (Data Access Layer - db.py).....	54
2. Διαχείριση Ερωτημάτων (Query Repository - queries.py).....	54
3. Επίπεδο Παρουσίασης και Δρομολόγησης (Presentation Layer - app.py).....	54
Δ. Χρήση AI/LLMs:.....	55

A. Σχεδιασμός και υλοποίηση Βάσης Δεδομένων

Η ανάπτυξη της βάσης ακολούθησε διαδοχικά στάδια: εννοιολογική μοντελοποίηση (conceptual modeling), σχεσιακή μετατροπή (relational modeling), υλοποίηση βασικών περιορισμών (constraints), υλοποίηση σύνθετων επιχειρησιακών κανόνων με triggers και stored procedures, ορισμό views και τελική φόρτωση δεδομένων.

A.1. Διάγραμμα Οντοτήτων – Συσχετίσεων (Entity-Relationship Diagram)

A.1.1 Ανάλυση Οντοτήτων (Entities)

Στο παρόν στάδιο της εννοιολογικής μοντελοποίησης, το εξαιρετικά σύνθετο περιβάλλον του Νοσοκομείου "Υγειόπολης" αναλύθηκε και αποτυπώθηκε σε ένα Διάγραμμα Οντοτήτων-Συσχετίσεων (ER Diagram). Η σχεδιαστική μας προσέγγιση διατηρεί τις οντότητες που διαθέτουν ανεξάρτητο κύκλο ζωής ως ισχυρές οντότητες (strong entities), ενώ μεταφέρει πλήρως εξαρτώμενες πληροφορίες που η ύπαρξή τους συνδέεται άρρηκτα με κάποια άλλη οντότητα σε ασθενείς οντότητες (weak entities). Παρακάτω αναλύονται διεξοδικά οι κεντρικές οντότητες, οι συσχετίσεις τους, καθώς και οι επιχειρησιακοί κανόνες (business rules) που επιβάλλουν τους αντίστοιχους περιορισμούς πληθικότητας (cardinality) και συμμετοχής (participation).

Σημείωση: Στο ER Diagram τα discriminators των weak entities έχουν επισημανθεί με μορφή: *discriminator* (*italics+underline*), ενώ τα primary keys των string entities με μορφή: primary key (*underline*). Η συγκεκριμένη επιλογή έγινε διότι το λογισμικό σχεδίασης που χρησιμοποιήσαμε (draw.io) δεν έδινε την δυνατότητα για διακεκομμένη υπογράμμιση των discriminators.

Entities

Πίνακας	Σύντομη περιγραφή
Staff	Κοινά στοιχεία όλου του προσωπικού.
Doctor	Ειδικά στοιχεία ιατρών.
Department	Τμήματα νοσοκομείου.
Nurse	Ειδικά στοιχεία νοσηλευτών.
AdminStaff	Ειδικά στοιχεία διοικητικού προσωπικού.
Room	Δωμάτια/κλίνες ανά τμήμα. (Weak entity του Department)
InsuranceProvider	Ασφαλιστικοί φορείς.
Patient	Βασικά στοιχεία ασθενών.
EmergencyContact	Επαφές ανάγκης ασθενών. (Weak entity του Patient)
Cost	ΚΕΝ κόστη και προβλεπόμενη διάρκεια νοσηλείας.
Hospitalization	Νοσηλείες ασθενών. (Weak entity του Patient)
TriageEvent	Περιστατικά διαλογής στα επείγοντα.
HospEvaluation	Αξιολογήσεις νοσηλίων από ασθενείς.

Diagnosis	Κατάλογος διαγνώσεων ICD.
LabTest	Κατάλογος εργαστηριακών εξετάσεων.
HospLabTest	Γεγονός ότι ένας ασθενής έκανε μια εργαστηριακή εξέταση
ProcedureType	Κατάλογος ιατρικών πράξεων.
ProcedureRoom	Αίθουσες ιατρικών πράξεων.
ProcedureEvent	Συγκεκριμένα συμβάντα ιατρικών πράξεων.
ShiftType	Τύποι βαρδιών και ωράρια.
Shift	Βάρδιες ανά τμήμα και ημερομηνία.
DrugType	Κατάλογος φαρμάκων.
Substances	Δραστικές ουσίες φαρμάκων.
PrescriptionEvent	Συνταγογραφήσεις φαρμάκων.
Image	Εικόνες για προσωπικό, τμήματα, δωμάτια ή αίθουσες πράξεων.

A.1.2 Σχόλια για την μοντελοποίηση των οντοτήτων

1. Ιεραρχίες και Εξειδίκευση (Specialization / Generalization)

Staff – Doctor / Nurse / AdminStaff: Η οντότητα Staff λειτουργεί ως υπερκλάση (superclass) για όλο το προσωπικό, περιέχοντας τα κοινά γνωρίσματα (AMKA, Name, Contact Info). Οι οντότητες Doctor, Nurse και AdminStaff αποτελούν τις υποκλάσεις (subclasses) στις οποίες μεταφέρονται τα εξειδικευμένα πεδία (π.χ. το License και το Specialty για τους γιατρούς). Η συγκεκριμένη ιεραρχία μοντελοποιήθηκε με περιορισμούς Total και Disjoint (Ολική και Ξένη Συμμετοχή). Αυτό διασφαλίζει ότι κάθε μέλος του προσωπικού πρέπει να ανήκει σε μία (Total) και μόνο μία (Disjoint) από τις τρεις υπο-κατηγορίες, εξασφαλίζοντας την ακεραιότητα των δεδομένων προσωπικού.

2. Μοντελοποίηση Γεγονότων ως Κεντρικές Οντότητες (Event Entities vs. N-ary Relationships)

Για την αποτύπωση πολύπλοκων ιατρικών και επιχειρησιακών συμβάντων του νοσοκομείου, όπως οι ιατρικές πράξεις/επεμβάσεις (ProcedureEvent), η διαλογή στα επείγοντα (TriageEvent), οι εργαστηριακές εξετάσεις που μπορεί να κάνει ένας ασθενής κατά τη διάρκεια της νοσηλείας του (HospLabTest) και οι συνταγογραφήσεις (PrescriptionEvent), επιλέχθηκε η στρατηγική της "ανύψωσης" (promotion) των γεγονότων σε αυτόνομες οντότητες. Σε όσες από αυτές η οντότητα ενέπλεκε παραπάνω από δύο άλλες οντότητες, θεωρήθηκε βέλτιστο να εισαχθεί ένα νέο χαρακτηριστικό (attribute) ταυτοποίησης με τη μορφή τεχνητού κλειδιού (surrogate key / ID), μετατρέποντάς τις σε ισχυρές οντότητες, ενώ άλλες μοντελοποιήθηκαν ως ασθενείς οντότητες (weak entities) με τους αντίστοιχους διαχωριστές (discriminators). Η αυστηρή σχεσιακή θεωρία επισημαίνει ότι κάθε πολυδιάστατη (n-ary) συσχέτιση μπορεί —και συχνά είναι επιβεβλημένο— να αναλυθεί σε μια κεντρική οντότητα γεγονότος που συνδέεται μέσω δυαδικών σχέσεων με τις συμμετέχουσες οντότητες. Αυτή η αρχιτεκτονική επιλογή προσδίδει στο γεγονός τον δικό του κύκλο ζωής (lifecycle) και επιτρέπει την εκχώρηση δικών του, εγγενών γνωρισμάτων (όπως ProcEventID, DateTime,

Dosage), λύνοντας το πρόβλημα της πολυπλοκότητας και των περιορισμών που θα προέκυπταν αν επιχειρούσαμε να ενώσουμε, για παράδειγμα, τον Ιατρό, τον Ασθενή, την Αίθουσα και το Φάρμακο σε μια δυσκίνητη τετραδική (quaternary) συσχέτιση.

3. Ασθενείς Οντότητες και Υπαρξιακή Εξάρτηση (Weak Entities & Existence Dependency)

Οντότητες όπως οι κλίνες (Room), οι νοσηλείες (Hospitalization), οι επαφές έκτακτης ανάγκης (EmergencyContact) και οι Αξιολογήσεις (Evaluation), αναγνωρίστηκαν και μοντελοποιήθηκαν αυστηρά ως Ασθενείς Οντότητες (Weak Entities). Μια ασθενής οντότητα είναι από τη φύση της υπαρξιακά εξαρτώμενη (existence dependent) από μια ισχυρή οντότητα και στερείται επαρκών γνωρισμάτων για τον σχηματισμό ενός ανεξάρτητου, καθολικού πρωτεύοντος κλειδιού. Για παράδειγμα, η οντότητα Room ταυτοποιείται πλήρως μόνο μέσω του συνδυασμού του τοπικού της αναγνωριστικού (διαχωριστής / discriminator) και του τμήματος στο οποίο ανήκει. Η επιλογή αυτή εξασφαλίζει ότι το conceptual μοντέλο αντικατοπτρίζει πιστά τη φυσική πραγματικότητα: ένα δωμάτιο νοσοκομείου ή μια νοσηλεία δεν υφίστανται εννοιολογικά στο κενό, παρά μόνο ως αναπόσπαστα τμήματα ενός Τμήματος ή ενός Ασθενούς αντίστοιχα.

4. Οντότητες - Κατάλογοι Αναφοράς για Σημασιολογική Ακεραιότητα (Reference / Catalog Entities)

Στο σχήμα ενσωματώθηκαν αυτόνομες οντότητες που λειτουργούν ως στατικοί κατάλογοι αναφοράς, όπως οι Διαγνώσεις (Diagnosis βάσει ICD-10), τα Κόστη (Cost βάσει KEN) και τα Φάρμακα (DrugType βάσει EMA). Η απόφαση να μοντελοποιηθούν ως διακριτές οντότητες (αντί για απλά πεδία κειμένου μέσα στη νοσηλεία ή στη συνταγή) αποτελεί θεμελιώδη κανόνα της κανονικοποίησης (normalization). Αποτρέπει την ανωμαλία εισαγωγής (insertion anomalies) και την πλεονασματική αποθήκευση, ενώ παράλληλα εγγυάται την απόλυτη ακεραιότητα πεδίου τιμών (domain integrity), καθώς το σύστημα δέχεται μόνο τις τυποποιημένες, επίσημες καταχωρήσεις του Υπουργείου Υγείας.

5. Κοινόχρηστες Οντότητες Πολυμέσων (Polymorphic/Shared Entities)

Για την ικανοποίηση της απαίτησης διαχείρισης πολυμέσων, σχεδιάστηκε μία ενιαία κοινόχρηστη οντότητα Image. Αντί να κατακερματιστεί το μοντέλο δημιουργώντας ξεχωριστές οντότητες εικόνων για κάθε πιθανό στόχο (π.χ. StaffImage, DepartmentImage, RoomImage), η οντότητα αφαιρέθηκε (abstracted) στο υψηλότερο δυνατό επίπεδο. Αυτό το μοτίβο σχεδίασης μειώνει τον αριθμό των πινάκων, αποφεύγει τη σπατάλη χώρου και παρέχει τεράστια επεκτασιμότητα: αν στο μέλλον το νοσοκομείο αποφασίσει να τηρεί φωτογραφίες ιατρικών μηχανημάτων, ο βασικός πυρήνας της οντότητας Image δεν θα χρειαστεί καμία απολύτως τροποποίηση.

A.1.3 Ανάλυση Συσχετίσεων (Relationships)

Λόγω της υψηλής πολυπλοκότητας του συστήματος και του μεγάλου πλήθους των συσχετίσεων, επιλέχθηκε η θεματική ομαδοποίησή τους ανά επιχειρησιακό υποσύστημα (business domain). Για κάθε συσχέτιση παρατίθεται ο Λόγος Πληθικότητας (Cardinality Ratio) και οι Περιορισμοί Συμμετοχής (Participation Constraints).

1. Υποσύστημα Ανθρώπινου Δυναμικού & Δομής (HR & Infrastructure)

Σε αυτή την ενότητα αποτυπώνονται οι οργανωτικές δομές, η ιεραρχία των γιατρών και η κατανομή του προσωπικού στα τμήματα.

Όνομα Συσχέτισης	Οντότητες & Πληθικότητα	Συμμετοχή	Περιγραφή / Σχεδιαστική Λογική
hasSupervisor	Doctor → Doctor (1:N)	Partial - Partial	Αναδρομική (recursive) σχέση. Ένας επόπτης επιβλέπει πολλούς γιατρούς, αλλά ένας γιατρός έχει το πολύ έναν επόπτη.
hasDirector	Department → Doctor (1:1)	Total - Partial	Κάθε τμήμα διοικείται από ακριβώς έναν Διευθυντή (ολική για το τμήμα).
doct_dept	Doctor → Department (M:N)	Total - Total	Ένας γιατρός μπορεί να ανήκει σε πολλά τμήματα και αντίστροφα.
nurse_dept	Nurse → Department (N:1)	Total - Partial	Κάθε νοσηλεύτης ανήκει σε ακριβώς ένα τμήμα.
admin_dept	AdminStaff → Department (N:1)	Total - Partial	Κάθε διοικητικός υπάλληλος ανήκει σε ακριβώς ένα τμήμα.
hasRoom	Department → Room (1:N)	Total - Total	Identifying Relationship για την ασθενή οντότητα Room. Ο αριθμός δωματίου είναι μοναδικός μόνο τοπικά στο τμήμα.
DoctorEvaluation Attr:(QoDoctorS)	Hospitalization → Doctor (M:N)	Partial - Partial	Δυνατότητα ενός ασθενούς να αξιολογήσει πολλαπλούς ιατρούς μετά την ολοκλήρωση της νοσηλείας του.

2. Υποσύστημα Ασθενών & Νοσηλείας (Patient Care & Hospitalization)

Εδώ περιλαμβάνονται οι ροές από την εισαγωγή (ή τη διαλογή στα επείγοντα) μέχρι το εξιτήριο και την αξιολόγηση.

Όνομα Συσχέτισης	Οντότητες & Πληθικότητα	Συμμετοχή	Περιγραφή / Σχεδιαστική Λογική
has_insurance	Patient → InsuranceProvider (N:1)	Total - Total	Κάθε ασθενής καλύπτεται από έναν ασφαλιστικό φορέα.

has_contact Attr:(Relation)	Patient → EmergencyContact (1:N)	Partial - Total	Identifying Relationship για την επαφή έκτακτης ανάγκης (Ασθενής Οντότητα).
admits	Patient → Hospitalization (1:N)	Partial - Total	Identifying Relationship. Η νοσηλεία εξαρτάται υπαρξιακά από τον ασθενή.
room_hosp	Hospitalization → Room (N:1)	Total - Partial	Η κλίνη στην οποία πραγματοποιείται η νοσηλεία.
has_cost	Hospitalization → Cost (N:1)	Total - Partial	Κάθε νοσηλεία αντιστοιχίζεται σε έναν ΚΕΝ κωδικό κοστολόγησης.
has_eval	Hospitalization → HospEvaluation (1:1)	Partial - Total	Identifying Relationship. Η αξιολόγηση υφίσταται μόνο αν υπάρχει ολοκληρωμένη νοσηλεία.
admitted_with	Hospitalization → Diagnosis (M:N)	Total - Partial	Διασύνδεση με τον κατάλογο ICD-10 για τις διαγνώσεις κατά την εισαγωγή.
exited_with	Hospitalization → Diagnosis (M:N)	Total - Partial	Διασύνδεση με τον κατάλογο ICD-10 για τις τελικές διαγνώσεις εξιτηρίου.

3. Υποσύστημα Ιατρικών Πράξεων & Εξετάσεων (Medical Procedures & Labs)

Καλύπτει τα δυναμικά ιατρικά συμβάντα που διενεργούνται στο νοσοκομείο.

Όνομα Συσχέτισης	Οντότητες & Πληθικότητα	Συμμετοχή	Περιγραφή / Σχεδιαστική Λογική
has_test	HospLabTest → LabTest (N:1)	Total - Partial	Κάθε event εργαστηριακής εξέτασης αντιστοιχεί σε ένα μοναδικό είδος εξέτασης.
has_hosp	Hospitalization → HospLabTest	Partial - Total	Κάθε event εργαστηριακής εξέτασης αντιστοιχεί σε μία μοναδική νοσηλεία.
ordered_by	HospLabTest → Doctor (N:1)	Total - Partial	Δείχνει ποιος γιατρός παρήγγειλε τη συγκεκριμένη εργαστηριακή εξέταση.
happened_in	ProcedureEvent → ProcedureType (N:1)	Total - Partial	Το συμβάν της ιατρικής πράξης αντιστοιχίζεται στον επίσημο κατάλογο πράξεων.
used_in	ProcedureEvent → ProcedureRoom (N:1)	Total - Partial	Η αίθουσα (π.χ. χειρουργείο) όπου εκτελείται η πράξη.

operates_in	ProcedureEvent → Doctor (M:N)	Total - Partial	Οι κύριοι και βοηθοί χειρουργοί που συμμετέχουν στην πράξη.
assists_in	ProcedureEvent → Nurse (M:N)	Partial - Partial	Η υποστηρικτική νοσηλευτική ομάδα του χειρουργείου.
helps_in	ProcedureEvent → AdminStaff (M:N)	Partial - Partial	Υποστήριξη διοικητικού προσωπικού για μια συγκεκριμένη ιατρική πράξη.
mainDoc	ProcedureEvent → Doctor (N:1)	Total - Partial	Δείχνει τον κύριο γιατρό που επιμελείται την συγκεκριμένη ιατρική πράξη

4. Υποσύστημα Διαχείρισης Επειγόντων, Φαρμάκων & Βαρδιών

Όνομα Συσχέτισης	Οντότητες & Πληθικότητα	Συμμετοχή	Περιγραφή / Σχεδιαστική Λογική
has_patient	TriageEvent → Patient (N:1)	Total - Total	Ο ασθενής που προσέρχεται στη διαλογή επειγόντων.
referred_to	TriageEvent → Hospitalization (1:1)	Partial - Total	Προαιρετική σχέση. Υφίσταται μόνο αν η διαλογή οδηγήσει σε εισαγωγή.
allergic_to	Patient → Substances (M:N)	Partial - Partial	Οι ουσίες στις οποίες ένας ασθενής έχει αλλεργία.
hasSubstances	DrugType → Substances (M:N)	Total - Partial	Η αντιστοίχιση των εγκεκριμένων φαρμάκων του EMA με τις δραστικές τους ουσίες.
has_drug	PrescriptionEvent → DrugType (N:1)	Total - Partial	Το συγκεκριμένο φάρμακο που συνταγογραφείται σε ένα συμβάν συνταγογράφησης.
in_hosp	PrescriptionEvent → Hospitalization (N:1)	Total - Partial	Η νοσηλεία στο πλαίσιο της οποίας λαμβάνει χώρα η συνταγογράφηση.
has_doctor	PrescriptionEvent → Doctor (N:1)	Total - Partial	Ο ιατρός που εκδίδει και υπογράφει τη συγκεκριμένη συνταγή.
belongs_to	Shift → Department (N:1)	Total - Total	Η βάρδια εξαρτάται υπαρξιακά από το τμήμα για το οποίο προγραμματίζεται.
has_type	Shift → ShiftType (N:1)	Total - Total	Identifying Relationship. Η κάθε βάρδια ορίζεται και εξαρτάται από την ζώνη ώρας.

has_doctor / has_nurse / has_admin	Shift → Doctor / Nurse / AdminStaff (M:N) Εδώ πάνω στη σχέση έχουν εισαχθεί και τα ακριβή όρια που επιβάλλουν οι επιχειρησιακοί κανόνες της εργασίας (π.χ. 6..* -> τουλάχιστον 6 νοσηλευτές στη βάρδια)	Partial - Partial	Τρεις διακριτές συσχετίσεις που ενώνουν τη βάρδια με το προσωπικό. Χρησιμοποιούνται για τον έλεγχο της ελάχιστης στελέχωσης.
Exclusive Arc	Image → Staff / Dept / Room / ProcedureRoom (1:1 ή 1:N)	Partial - Total	Κοινή οντότητα πολυμέσων. Η εικόνα αντιστοιχίζεται αποκλειστικά σε έναν και μόνο έναν επιτρεπτό στόχο κάθε φορά.

A.2. Σχεσιακό Διάγραμμα και υλοποίηση Βάσης Δεδομένων (Relational Model)

A.2.1. Μετάβαση από το ER στο Relational

Η μετάπτωση (mapping) από το Εννοιολογικό Διάγραμμα (Entity-Relationship) στο Σχεσιακό Μοντέλο (Relational Model) αποτελεί ένα κρίσιμο στάδιο του λογικού σχεδιασμού (logical database design). Κατά τη διαδικασία αυτή, εφαρμόστηκαν αυστηρά οι καθιερωμένοι αλγόριθμοι μετασχηματισμού της σχεσιακής θεωρίας (ER-to-Relational Mapping Algorithm), ώστε να εξασφαλιστεί η εξάλειψη του πλεονασμού (redundancy) και η ορθή κανονικοποίηση του σχήματος (normalization).

Οι βασικοί κανόνες μετάπτωσης που εφαρμόστηκαν στο σύστημα του "HospitalDB" είναι οι εξής:

1. Ισχυρές Οντότητες (Strong Entities): Κάθε ισχυρή οντότητα μετατράπηκε σε ανεξάρτητο πίνακα, περιλαμβάνοντας αποκλειστικά τα απλά γνωρίσματά της. Επιλέχθηκε το βέλτιστο υπονήγιο κλειδί ως Πρωτεύον Κλειδί (Primary Key).
2. Ιεραρχίες Εξειδίκευσης (Specialization / Generalization): Για την ιεραρχία του προσωπικού, επιλέχθηκε η στρατηγική της διατήρησης της υπερκλάσης και των υποκλάσεων σε ξεχωριστούς πίνακες (Multiple relations—superclass and subclasses). Δημιουργήθηκε ο κεντρικός πίνακας Staff (με PK το AMKA), ενώ οι υποκλάσεις Doctor, Nurse και AdminStaff δημιουργήθηκαν ως ανεξάρτητοι πίνακες, όπου το AMKA λειτουργεί ταυτόχρονα ως Πρωτεύον Κλειδί (Primary Key) και ως Ξένο Κλειδί (Foreign Key) προς τον πίνακα Staff.
3. Ασθενείς Οντότητες και Οντότητες - Γεγονότα (Weak Entities & Event Entities): Ενώ θεωρητικά το πρωτεύον κλειδί μιας ασθενούς οντότητας προκύπτει από τον συνδυασμό του ξένου κλειδιού της ισχυρής οντότητας στην οποία ανήκει και του δικού της διαχωριστή (partial key), για λόγους βελτιστοποίησης της απόδοσης (performance optimization) και ευκολίας στις συσχετίσεις, σε πολλά γεγονότα (όπως Hospitalization, TriageEvent, ProcedureEvent) εισήχθησαν τεχνητά κλειδιά (Surrogate Keys) μέσω της ιδιότητας AUTO_INCREMENT. (1)

4. Συσχετίσεις 1:1 και 1:N: Για τις δυαδικές συσχετίσεις αυτών των τύπων, ακολουθήθηκε η προσέγγιση του Ξένου Κλειδιού (Foreign Key approach). Το πρωτεύον κλειδί της πλευράς "I" εισήχθη ως ξένο κλειδί στον πίνακα της πλευράς "N". Στις 1:1 σχέσεις, όπως αυτή μεταξύ Department και Διευθυντή ιατρού (hasDirector), το ξένο κλειδί (DirectorAMK) τοποθετήθηκε στην οντότητα με την Ολική Συμμετοχή (δηλαδή στο Department). Αυτό γίνεται για να μην δημιουργηθούν NULL τιμές.
5. Συσχετίσεις M:N (Πολλά προς Πολλά): Για κάθε συσχέτιση M:N δημιουργήθηκε ένας ενδιάμεσος πίνακας συσχέτισης (Relationship relation / Junction table), ο οποίος φιλοξενεί ως ξένα κλειδιά τα πρωτεύοντα κλειδιά των δύο συμμετεχουσών οντοτήτων. Τέτοιοι πίνακες στο σχήμα μας είναι οι operates_in, assists_in, allergic_to, και οι πίνακες στελέχωσης βαρδιών (hasDoctor κ.λπ.).
6. Multivalued Attributes: Τα multivalued attributes αποσπάστηκαν από τις κύριες οντότητες και αποτέλεσαν νέους, αυτόνομους πίνακες, στους οποίους τοποθετήθηκε ως ξένο κλειδί το αναγνωριστικό της γονικής οντότητας. Με αυτή τη μέθοδο υλοποιήθηκαν οι πίνακες StaffPhone, PatientPhone, EmContactPhone και DrugSupportPhone.

(1)

Στο σχεσιακό σύστημα διαχείρισης MySQL και ειδικότερα στη μηχανή αποθήκευσης InnoDB, η αρχιτεκτονική των ευρετηρίων (indexes) αλληλεπιδρά άμεσα με την επιλογή του Πρωτεύοντος Κλειδιού (Primary Key), επηρεάζοντας δραματικά το κόστος I/O και την κατανάλωση μνήμης.

Παρακάτω αναλύεται ο μηχανισμός λειτουργίας των ευρετηρίων και τα πλεονεκτήματα που προκύπτουν από τη χρήση των surrogate keys:

1. Clustered Indexes και B+ Trees

Στη μηχανή InnoDB, κάθε πίνακας οργανώνεται φυσικά στον δίσκο γύρω από το πρωτεύον κλειδί του, σχηματίζοντας ένα Clustered Index μορφής B+-Tree. Σε αυτό το δέντρο, οι κόμβοι-φύλλα (leaf nodes) περιέχουν τα πραγματικά δεδομένα ολόκληρης της γραμμής (file organization index). Όταν χρησιμοποιείται ένα φυσικό, σύνθετο κλειδί (π.χ. συνδυασμός ΑΜΚΑ ασθενούς και Ημερομηνίας/Ωρας) ως πρωτεύον κλειδί, οι νέες εγγραφές εισάγονται τυχαία (random inserts) μέσα στη δομή του B+Tree. Αυτό αναγκάζει τη μηχανή να εκτελεί χρονοβόρους "επιμερισμούς σελίδων" (page splits), δημιουργώντας κατακερματισμό (fragmentation) και αφήνοντας τις σελίδες δεδομένων (data pages) μισογεμάτες. Αντίθετα, με τη χρήση ενός ακέραιου, μονοτονικά αυξανόμενου surrogate key (AUTO_INCREMENT), οι νέες εγγραφές προσαρτώνται πάντα στο τέλος του δέντρου (sequential inserts). Αυτό διασφαλίζει ότι οι σελίδες των 16KB γεμίζουν πλήρως και σειριακά, εξαλείφοντας τον κατακερματισμό, βελτιώνοντας την τοπικότητα της κρυφής μνήμης (cache locality) και μειώνοντας δραματικά τον αριθμό των ακριβών εγγραφών στον δίσκο.

2. Μείωση Μεγέθους στα Δευτερεύοντα Ευρετήρια (Secondary Indexes Bloat)

Πέρα από το πρωτεύον κλειδί, η βάση χρησιμοποιεί Δευτερεύοντα Ευρετήρια (Secondary Indexes) για την επιτάχυνση των αναζητήσεων σε άλλα πεδία (π.χ. αναζήτηση νοσηλίων ανά DepartmentID ή ανά CostCode). Στη μηχανή InnoDB, οι κόμβοι-φύλλα των δευτερευόντων ευρετηρίων δεν περιέχουν φυσικούς δείκτες (pointers) προς τα δεδομένα, αλλά αποθηκεύουν αντίγραφα της τιμής του πρωτεύοντος κλειδιού (αυτό συμβαίνει για λόγους απόδοσης, όπως ψάξαμε από το documentation της MySQL). Αν το πρωτεύον κλειδί της Νοσηλείας ήταν ένα βαρύ σύνθετο κλειδί (π.χ. VARCHAR και DATETIME), αυτή η τεράστια τιμή θα αντιγραφόταν μέσα σε κάθε εγγραφή κάθε δευτερεύοντος ευρετηρίου. Χρησιμοποιώντας ένα μικρό ακέραιο κλειδί (surrogate key τύπου INT), διατηρούμε το

μέγεθος (footprint) όλων των δευτερευόντων ευρετηρίων στο ελάχιστο, επιταχύνοντας τις σαρώσεις και εξοικονομώντας τεράστιο αποθηκευτικό χώρο.

3. Μεγιστοποίηση της Απόδοσης στην Κύρια Μνήμη (InnoDB Buffer Pool)

Ο πιο κρίσιμος παράγοντας απόδοσης στη MySQL είναι το Buffer Pool, η περιοχή της κύριας μνήμης (RAM) που αποθηκεύονται προσωρινά οι σελίδες δεδομένων και τα ευρετήρια για να αποφευχθούν οι προσπελάσεις (I/O) στον αργό δίσκο. Διατηρώντας τα πρωτεύοντα και δευτερεύοντα κλειδιά όσο το δυνατόν πιο συμπαγή μέσω των surrogate keys, αυξάνεται ο δείκτης διακλάδωσης (fan-out) των B+ Trees. Αυτό σημαίνει ότι περισσότερες εγγραφές ευρετηρίου χωρούν σε μία μόνο σελίδα (page) μνήμης. Κατά συνέπεια, μεγαλύτερο μέρος του δέντρου χωράει (cached) εντός του Buffer Pool, μεγιστοποιώντας τον δείκτη επιτυχίας (cache hit ratio) και επιτρέποντας στο σύστημα να απαντά σε ερωτήματα με ταχύτητες κύριας μνήμης, αντί για ταχύτητες δίσκου.

4. Αποδοτικότητα στις Πράξεις Συνένωσης (Join Performance)

Το σχεσιακό μας μοντέλο απαιτεί εκατοντάδες πράξεις συνένωσης (JOINS) μεταξύ των γεγονότων (events) και των διαστάσεων τους (π.χ. Hospitalization με Patient, ProcedureEvent και HospLabTest). Οι αλγόριθμοι Nested-Loop Join και Hash Join, τους οποίους χρησιμοποιεί ο Query Optimizer, εκτελούνται πολύ ταχύτερα όταν συγκρίνουν πεδία ακεραίων (όπως τα surrogate keys) παρά όταν αναγκάζονται να κάνουν συγκρίσεις αλφαριθμητικών (strings) ή αξιολογήσεις πολλαπλών στηλών σε σύνθετα κλειδιά.

Με βάση τους παραπάνω κανόνες συντάξαμε τον κώδικα της SQL στο αρχείο install.sql που δημιουργεί το schema της βάσης μας. Το τρέχον install.sql δημιουργεί 41 πίνακες.

Κανονικοποίηση (Normalization)

Αν και το γενικό σχεσιακό σχήμα σχεδιάστηκε με γνώμονα την πλήρη συμμόρφωση με την Τρίτη Κανονική Μορφή (3NF), προχωρήσαμε σε συγκεκριμένες, αυστηρά ελεγχόμενες σχεδιαστικές αποκλίσεις στο φυσικό επίπεδο (physical level) για τη βελτιστοποίηση της απόδοσης και την ικανοποίηση επιχειρησιακών κανόνων:

1. Ενσωμάτωση Derived Attributes: Γνωρίσματα όπως τα ActualDays, ChargePerDay και EndTime αποτελούν παραγόμενη πληροφορία. Για να αποφευχθεί η παραβίαση της 3NF (που απαγορεύει την αποθήκευση υπολογίσιμων πεδίων προς αποφυγή update anomalies), υλοποιήθηκαν στη MySQL αποκλειστικά ως GENERATED ALWAYS AS ... VIRTUAL. Με τον τρόπο αυτό, δεν καταλαμβάνουν χώρο στον δίσκο και εγγυώνται απόλυτη συνέπεια, δρώντας ως in-table views.
2. Subtype Discriminator: Στον πίνακα Staff, η ύπαρξη του πεδίου Type αποτελεί ελεγχόμενη πλεονασματικότητα. Βάσει της σχεσιακής θεωρίας, η χρήση ενός πεδίου τύπου στην υπερκλάση για ιεραρχίες τύπου Disjoint επιταχύνει δραματικά τα ερωτήματα, καθώς το σύστημα γνωρίζει εκ των προτέρων σε ποιον υπο-πίνακα (Doctor, Nurse, AdminStaff) πρέπει να κατευθύνει το JOIN.
3. Ελεγχόμενη Μεταβατική Εξάρτηση στον Πίνακα TriageEvent: Ο πίνακας TriageEvent περιέχει τόσο το PatientAMKA όσο και το HospitalizationID. Ενώ θεωρητικά σε περιπτώσεις εισαγωγής (Accepted) το PatientAMKA θα μπορούσε να ανακτηθεί μέσω του HospitalizationID (μεταβατική εξάρτηση), η παρουσία του κρίνεται υπαρκτά αναγκαία,

καθώς το HospitalizationID είναι NULLable (για Pending/Discarded περιστατικά). Αυτή η εγγενής πλεονασματικότητα ελέγχεται πλήρως και αυστηρά μέσω των Triggers (trg_triage_hospitalization_same_patient_insert/update), τα οποία εγγυώνται ότι δεν θα υπάρξει ποτέ ασυνέπεια μεταξύ των πινάκων

A.2.2. Derived Attributes

Στη θεωρία του Σχεσιακού Μοντέλου Δεδομένων, ένα παραγόμενο χαρακτηριστικό είναι ένα πεδίο του οποίου η τιμή δεν αποθηκεύεται στατικά, αλλά προκύπτει δυναμικά από τις τιμές άλλων, συσχετιζόμενων χαρακτηριστικών ή οντοτήτων της βάσης. Όπως αναφέραμε στην κανονικοποίηση, αποφύγαμε τη στατική αποθήκευση αυτών των πεδίων, προκειμένου να διασφαλιστεί η Τρίτη Κανονική Μορφή (3NF) και να αποτραπούν φαινόμενα ασυνέπειας.

Οι κύριες επιλογές παραγόμενων χαρακτηριστικών στο σχήμα μας είναι οι εξής:

1. **Πραγματικές Ημέρες Νοσηλείας (ActualDays - Πίνακας Hospitalization)** : Το πεδίο ActualDays ορίστηκε ως παραγόμενο, καθώς υπολογίζεται άμεσα από τη διαφορά του χρονικού στίγματος εισόδου και εξόδου (ExitDateTime - AdmissionDateTime). Το πεδίο υλοποιήθηκε ως *Virtual Generated Column* αξιοποιώντας τη συνάρτηση TIMESTAMPDIF. Αυτό εγγυάται μαθηματικά ότι αν τροποποιηθεί η ημερομηνία εξιτηρίου ενός ασθενούς, οι ημέρες νοσηλείας θα αναπροσαρμοστούν ακαριαία.
2. **Ωρα Λήξης Βάρδιας (EndTime - Πίνακας ShiftType)** : Βάσει των επιχειρησιακών κανόνων του νοσοκομείου, κάθε βάρδια προσωπικού έχει σταθερή διάρκεια 8 ωρών. Ως εκ τούτου, η ώρα λήξης (EndTime) είναι αυστηρά παραγόμενη από την ώρα έναρξης (StartTime). Στο σχήμα υλοποιήθηκε μέσω της εντολής **GENERATED AS ADDTIME(StartTime, '08:00:00')**. Η αρχιτεκτονική αυτή εμποδίζει οποιονδήποτε διαχειριστή να καταχωρήσει κατά λάθος μια βάρδια 7 ή 9 ωρών, επιβάλλοντας συνέπεια στα ωράρια.
3. **Ημερήσια Πρόσθετη Χρέωση (ExtraChargePerDay - Πίνακας Cost)**: Στο σύστημα των KEN, εάν η νοσηλεία ξεπεράσει την προβλεπόμενη ΜΔΝ, επιβάλλεται ημερήσια προσαύξηση. Η ημερήσια αυτή χρέωση είναι παραγόμενο χαρακτηριστικό, καθώς προκύπτει από τον τύπο $\text{BaseCost} / \text{PredictedAvgTime}$. Αποθηκεύοντάς το ως **GENERATED COLUMN** στον πίνακα Cost, γλιτώνουμε περιττό αποθηκευτικό χώρο και εξασφαλίζουμε ότι, αν το Υπουργείο Υγείας αλλάξει το Βασικό Κόστος ενός KEN, η ημερήσια προσαύξηση θα ενημερωθεί αυτόματα στο σύστημα.
4. **Συνολικά και Πρόσθετα Κόστη Νοσηλείας (AdditionalFees & TotalFees)** Το AdditionalFees παράγεται από τον τύπο $(\text{ActualDays} - \text{PredictedAvgDays}) *$

ExtraChargePerDay. Ομοίως, το TotalFees αποτελείται από το Βασικό Κόστος συν τα Πρόσθετα Τέλη, συν το άθροισμα όλων των φαρμάκων και εργαστηριακών εξετάσεων (HospLabTest, ProcedureEvent). Επειδή ο υπολογισμός τους απαιτεί πολύπλοκες συναθροίσεις δεδομένων από πολλαπλούς πίνακες, δεν υλοποιήθηκαν ως απλές εικονικές στήλες, αλλά υπολογίζονται δυναμικά και με ασφάλεια από την **αποθηκευμένη διαδικασία CalculateHospitalizationBill**.

5. **Ηλικία Προσωπικού και Ασθενών (Age - Πίνακες Staff & Patient):** Η ηλικία δεν αποθηκεύεται ως φυσικό πεδίο, αλλά παράγεται δυναμικά από το αποθηκευμένο πεδίο της Ημερομηνίας Γέννησης (BirthDate). Σε αντίθεση με το παραγόμενο πεδίο ActualDays, το οποίο υπολογίζεται από δύο σταθερές ημερομηνίες (AdmissionDateTime και ExitDateTime) και άρα επιτρέπεται να οριστεί ως VIRTUAL GENERATED COLUMN, η ηλικία εξαρτάται από το ρολόι του συστήματος. Επειδή η συνάρτηση CURDATE() είναι μη-ντετερμινιστική, η μηχανή της MySQL δεν επιτρέπει τη χρήση της σε εικονικές στήλες πίνακα. Για τον λόγο αυτό, η εξαγωγή της ηλικίας υλοποιήθηκε αρχιτεκτονικά στο επίπεδο των Όψεων. Όπως διακρίνεται στα views ActiveStaff, (& DocInfo) και ActivePatient το πεδίο Age υπολογίζεται δυναμικά μέσω της συνάρτησης TIMESTAMPDIFF(YEAR, BirthDate, CURDATE()).
6. **Καταμέτρηση Κλινών ανά Τμήμα (RoomNum – Όψη `DepartmentInfo`):** Το συγκεκριμένο παραγόμενο χαρακτηριστικό υλοποιήθηκε αποκλειστικά μέσω της Όψης **DepartmentInfo**, εξασφαλίζοντας δυναμική συνάθροιση των δεδομένων. Ο σχεδιασμός της όψης βασίζεται σε δύο θεμελιώδεις τεχνικές αρχές:
 - a. Εφαρμογή **LEFT JOIN**: Η σύνδεση του πίνακα τμημάτων με τον πίνακα Room πραγματοποιείται μέσω **LEFT JOIN** αντί για εσωτερική συνένωση. Η επιλογή αυτή εγγυάται ότι ακόμη και ένα τμήμα χωρίς εγγεγραμμένες κλίνες θα συμπεριληφθεί κανονικά στα αποτελέσματα, διατηρώντας την πληρότητα της πληροφορίας.
 - b. Ακριβής Συνάθροιση COUNT(r.ID): Η καταμέτρηση εκτελείται στοχευμένα επί του πρωτεύοντος κλειδιού *r.ID* και όχι μέσω της γενικής COUNT(*). Σε περιπτώσεις τμημάτων χωρίς κλίνες, η σχεσιακή μηχανή αναγνωρίζει τις NULL τιμές του πεδίου και επιστρέφει το ορθό ακέραιο 0..

A.2.3. Απαραίτητοι Περιορισμοί για ορθότητα

Ο βασικός σχεδιαστικός κανόνας που ακολουθήθηκε είναι η ελαχιστοποίηση της χρήσης triggers: κάθε κανόνας που μπορεί να εκφραστεί σε επίπεδο πεδίου ή γραμμής αποδίδεται με CHECK ή UNIQUE, και μόνο οι κανόνες που απαιτούν ανάγνωση ξένων πινάκων μεταφέρονται σε trigger. Αυτή η πρακτική εξασφαλίζει ότι το σχήμα παραμένει ευανάγνωστο και οι παραβιάσεις ακεραιότητας εντοπίζονται στο χαμηλότερο δυνατό επίπεδο, πριν αναλωθούν πόροι σε περαιτέρω επεξεργασία.

A.2.3.1 Table Level Constraints

1. Πίνακας Staff

Περιορισμός / SQL	Σκεπτικό
AMKA REGEXP '^[0-9]{11}\$'	Το AMKA είναι πάντα ακριβώς 11 αριθμητικά ψηφία. Ο κανόνας απορρίπτει κωδικούς με γράμματα, ειδικούς χαρακτήρες ή λανθασμένο μήκος.
Email UNIQUE	Κάθε μέλος προσωπικού διαθέτει διαφορετική διεύθυνση email. Αποτρέπει την κοινή χρήση λογαριασμού μεταξύ δύο υπαλλήλων.
Type IN ('Doctor', 'Nurse', 'AdminStaff')	Υλοποιεί τον περιορισμό Disjoint της ιεραρχίας: κάθε υπάλληλος ανήκει σε ακριβώς μία από τις τρεις επιτρεπτές κατηγορίες. Χρησιμοποιείται και προγραμματιστικά στις stored procedures για safety κατά τις εισαγωγές.
BirthDate < HireDate	Χρονική συνέπεια: κανείς δεν μπορεί να έχει προσληφθεί πριν γεννηθεί.

2. Πίνακας StaffPhone / PatientPhone / EmContactPhone

Περιορισμός / SQL	Σκεπτικό
Phone REGEXP '^[0-9]{10}\$'	10ψήφιος αριθμητικός κωδικός χωρίς πρόθεμα. Αποτρέπει τη λανθασμένη καταχώρηση αριθμών με κενά, παύλες ή μη αριθμητικούς χαρακτήρες. Τα τηλέφωνα των φαρμακευτικών εταιρειών (DrugSupportPhone) επιτρέπουν 10–14 ψηφία λόγω διεθνών αριθμών.

3. Πίνακας Doctor

Περιορισμός / SQL	Σκεπτικό
License UNIQUE	Ο αριθμός άδειας ιατρικού συλλόγου είναι εξ ορισμού μοναδικός. Εμποδίζει την καταχώρηση δύο ιατρών με κοινή άδεια.
Rank IN ('Resident', 'Director', 'Registrar', 'Consultant')	Επιβάλλει τις τέσσερις μόνο επιτρεπτές βαθμίδες της εκφώνησης (Ειδικευόμενος, Διευθυντής, Επιμελητής Β', Επιμελητής Α'). Οι αγγλικές ονομασίες χρησιμοποιήθηκαν για συνέπεια με τους ελέγχους στα triggers.

4. Πίνακας Department

Περιορισμός / SQL	Σκεπτικό
Name UNIQUE	Δεν επιτρέπεται να υπάρχουν δύο τμήματα με το ίδιο όνομα.

Floor >= 0	Ο αριθμός ορόφου δεν μπορεί να είναι αρνητικός. Αποτρέπει λάθη τυπογραφίας κατά την καταχώρηση θέσης τμήματος.
DirectorAMKA UNIQUE	Κάθε ιατρός μπορεί να είναι Διευθυντής το πολύ ενός τμήματος (σχέση 1:1 με ολική συμμετοχή από πλευράς τμήματος). Χωρίς αυτό, θα ήταν εφικτό ένας ιατρός να διοικεί πολλά τμήματα ταυτόχρονα.

5. Πίνακες Nurse και AdminStaff

Περιορισμός / SQL	Σκεπτικό
Rank IN ('AssistantNurse', 'Nurse', 'HeadNurse')	Περιορίζει τις βαθμίδες νοσηλευτή στις τρεις επιτρεπτές της εκφώνησης. Χρησιμεύει και ως βάση για την εφαρμογή διαφορετικών εφημεριακών κανόνων ανά βαθμίδα στα triggers.
UNIQUE(AMKA, DepartmentID)	Ένας νοσηλευτής/διοικητικός ανήκει ήδη σε ένα τμήμα. Το composite unique αποτρέπει διπλή καταχώρηση της ίδιας συσχέτισης και υποστηρίζει το FK των πινάκων βαρδίων (hasNurse, hasAdmin).

6. Πίνακας Room

Περιορισμός / SQL	Σκεπτικό
State IN ('Available', 'Occupied', 'Under Maintenance')	Οι τρεις μόνο επιτρεπτές καταστάσεις κλίνης. Εξασφαλίζει ότι το σύστημα γνωρίζει πάντα την ακριβή διαθεσιμότητα κάθε κλίνης.

7. Πίνακας Patient

Περιορισμός / SQL	Σκεπτικό
AMKA REGEXP '[0-9]{11}\$'	Ίδια λογική με το AMKA προσωπικού. Ο AMKA ασθενούς ακολουθεί τον ίδιο εθνικό κανόνα 11 ψηφίων.
Gender IN ('M', 'F')	Κωδικοποίηση φύλου με δύο επιτρεπτές τιμές, σύμφωνα με τις ανάγκες της βάσης. Εξαλείφει ασυνεπείς καταχωρήσεις (π.χ. 'male', 'MALE', 'A').
Weight > 0, Height > 0	Βάρος και ύψος δεν μπορούν να είναι μηδέν ή αρνητικά. Αποτρέπει default τιμές που θα αλλοίωναν υπολογισμούς όπως ο BMI.

8. Πίνακας Cost (KEN)

Περιορισμός / SQL	Σκεπτικό
-------------------	----------

BaseCost > 0	Δεν υπάρχει ΚΕΝ με μηδενικό ή αρνητικό βασικό κόστος — κάθε νοσήλιο έχει κάποια αξία.
PredictedAvgTime > 0	Η Μέση Διάρκεια Νοσηλείας (ΜΔΝ) πρέπει να είναι θετική. Χωρίς αυτό, ο τύπος ChargePerDay = BaseCost / PredictedAvgTime θα προκαλούσε διαίρεση με μηδέν.
ChargePerDay GENERATED AS (BaseCost / PredictedAvgTime)	Η ημερήσια επιπλέον χρέωση για υπέρβαση ΜΔΝ υπολογίζεται αυτόματα και αποθηκεύεται εικονικά.

9. Πίνακας Hospitalization

Περιορισμός / SQL	Σκεπτικό
UNIQUE(PatientAMKA, AdmissionDateTime)	Ένας ασθενής δεν μπορεί να εισαχθεί δύο φορές την ακριβώς ίδια χρονική στιγμή. Λόγω της χρήσης DATETIME (ακρίβεια 1 δευτερολέπτου).
ExitDateTime IS NULL OR ExitDateTime >= AdmissionDateTime	Χρονική συνέπεια νοσηλείας: η έξοδος δεν μπορεί να προηγείται της εισαγωγής. Επιτρέπει NULL για ανοιχτές (εν εξελίξει) νοσηλείες.
ActualDays GENERATED AS TIMESTAMPDIFF(...)	Ο αριθμός ημερών νοσηλείας υπολογίζεται αυτόματα από AdmissionDateTime και ExitDateTime. Η εισαγωγή ExitDateTime από τη DischargePatient ενεργοποιεί αμέσως τη διαθεσιμότητα του πεδίου για κοστολόγηση.

10. Πίνακας TriageEvent

Ο πίνακας TriageEvent έχει ιδιαίτερα σύνθετους περιορισμούς λόγω της φύσης του ως Event Table: κάθε περιστατικό βρίσκεται σε μία από τρεις καταστάσεις (Pending, Accepted, Discarded) και κάθε κατάσταση επιβάλλει διαφορετικό σύνολο υποχρεωτικών πεδίων.

Περιορισμός / SQL	Σκεπτικό
EmergencyLevel BETWEEN 1 AND 5	Τα επίπεδα επείγοντος της κλίμακας triage είναι αριθμοί 1 (άμεσο) έως 5 (μη επείγον). Εξαλείφει εκτός εύρους τιμές που θα αλλοίωναν τη σειρά προτεραιότητας.
Outcome IN ('Accepted', 'Discarded', 'Pending')	Ορίζει τις τρεις και μόνο επιτρεπτές καταστάσεις του κύκλου ζωής ενός triage περιστατικού.
UNIQUE(PatientAMKA, TriageDateTime)	Ο ίδιος ασθενής δεν μπορεί να καταχωρηθεί στο triage δύο φορές την ακριβώς ίδια στιγμή. Αποτρέπει διπλές εγγραφές λόγω προβλήματος ταυτόχρονης πρόσβασης.

chk_triage_outcome: Pending→(HospID IS NULL AND AssessmentDT IS NULL) Accepted→(HospID NOT NULL AND AssessmentDT NOT NULL) Discarded→(HospID IS NULL AND AssessmentDT NOT NULL)	Εδώ εξασφαλίζεται ότι κάθε συνδυασμός κατάστασης-πεδίων είναι λογικά συνεπής: ένα αποδεκτό περιστατικό πρέπει να συνδέεται με νοσηλεία, ένα απορριφθέν όχι, και ένα εκκρεμές δεν έχει ακόμη αξιολογηθεί.
---	--

11. Πίνακας HospEvaluation

Περιορισμός / SQL	Σκεπτικό
QoNurseS, Cleanliness, Food, GeneralExperience BETWEEN 1 AND 5	Κλίμακα Likert 1–5 για κάθε κριτήριο αξιολόγησης, σύμφωνα με την εκφώνηση. Εξαλείφει εκτός κλίμακας τιμές που θα αλλοίωναν τους μέσους όρους στο query Q4.

12. Πίνακας HospEvaluation

Περιορισμός / SQL	Σκεπτικό
QoDoctorS BETWEEN 1 AND 5	Και εδώ πάλι έχουμε κλίμακα Likert με between για την αξιολογήση (Quality of Service) για τους γιατρούς που έγραψαν συνταγή για έναν ασθενή.

13. Πίνακας HospLabTest

Η εργαστηριακή εξέταση στο πλαίσιο νοσηλείας μπορεί να βρίσκεται σε δύο ασύμβατες καταστάσεις: εκκρεμής (χωρίς αποτέλεσμα) ή ολοκληρωμένη (με αποτέλεσμα). Δύο συνεργαζόμενα CHECK constraints διασφαλίζουν αυτήν ακριβώς την αμοιβαία αποκλειστικότητα.

Περιορισμός / SQL	Σκεπτικό
UNIQUE(HospitalizationID, LabCode, LabDateTime)	Αποτρέπει την καταχώρηση δύο απολύτως πανομοιότυπων εξετάσεων (ίδια νοσηλεία, ίδιος κωδικός, ίδια ώρα). Επιτρέπει την επανάληψη της ίδιας εξέτασης σε διαφορετική χρονική στιγμή.
chk_pending_or_result: (PendingResult=0) OR (LabResult IS NOT NULL)	Μία ολοκληρωμένη εξέταση (PendingResult=0) πρέπει να διαθέτει αποτέλεσμα. Αποτρέπει εγγραφές ολοκληρωμένων εξετάσεων χωρίς τιμή.
chk_no_result_when_pending : (PendingResult=1) OR (LabResult IS NULL)	Μία εκκρεμής εξέταση δεν μπορεί να έχει ήδη αποτέλεσμα. Τα δύο CHECK λειτουργούν συμπληρωματικά διατηρώντας το integrity των εγγραφών των ιατρικών εξετάσεων.

14. Πίνακες ProcedureType, ProcedureRoom και ProcedureEvent

Περιορισμός / SQL	Σκεπτικό
ProcType IN ('Surgical', 'Diagnostic', 'Therapeutic')	Οι τρεις κατηγορίες ιατρικών πράξεων.
ProcDuration > 0	Η διάρκεια πράξης πρέπει να είναι θετική. Απαιτείται από τα triggers επικάλυψης procedures για τον υπολογισμό $EndTime = DateTime + ProcDuration$.
ProcRoomType IN ('OperatingRoom', 'InterventionRoom')	Οι δύο τύποι αιθουσών επεμβάσεων.
UNIQUE(ProcRoomID, DateTime)	Δύο επεμβάσεις δεν μπορούν να ξεκινούν ταυτόχρονα στην ίδια αίθουσα. Πρόκειται για ένα πρώτο γρήγορο φίλτρο — ο πλήρης έλεγχος χρονικής επικάλυψης (συμπεριλαμβανομένης της διάρκειας) ανατίθεται στα triggers trg_Procedure_Overlap.

15. Πίνακας ShiftType

Ο πίνακας αυτός ορίζει τα τρία σταθερά ωράρια βάρδιών και φιλοξενεί έναν βασικό περιορισμό: την αυτόματη δημιουργία EndTime.

Περιορισμός / SQL	Σκεπτικό
Name IN ('Morning', 'Afternoon', 'Night')	Τρεις επιτρεπτοί τύποι βάρδιας, σύμφωνα με το οκτάωρο σύστημα του νοσοκομείου.
StartTime IN ('07:00:00', '15:00:00', '23:00:00') + UNIQUE	Οι ώρες έναρξης είναι αυστηρά καθορισμένες. Το UNIQUE εξασφαλίζει ότι δεν μπορούν να υπάρξουν δύο βάρδιες με ίδια ώρα έναρξης.
chk_shift_type: Name='Morning'↔07:00 Name='Afternoon'↔15:00 Name='Night'↔23:00	Συνδυαστικός έλεγχος που εξασφαλίζει συνέπεια μεταξύ ονόματος βάρδιας και ώρας έναρξης. Αποτρέπει αντιφατικές εγγραφές όπως Name='Morning', StartTime='23:00:00'.
EndTime GENERATED AS ADDTIME(StartTime, '08:00:00')	Η ώρα λήξης βάρδιας υπολογίζεται αυτόματα ως StartTime + 8 ώρες. Αποτρέπει αναντιστοιχίες μεταξύ StartTime και EndTime και απλοποιεί τους υπολογισμούς στα triggers των εφημεριών.

16. Πίνακες hasDoctor, hasNurse, hasAdmin

Περιορισμός / SQL	Σκεπτικό
-------------------	----------

UNIQUE(DoctorAMKA, ShiftDate, ShiftTypeName)	Ο ίδιος ιατρός/νοσηλευτής/διοικητικό προσωπικό δεν μπορεί να εμφανίζεται δύο φορές στην ίδια βάρδια την ίδια ημέρα. Εξαλείφει διπλές καταχωρήσεις που θα αλλοίωναν τα count προσωπικού ανά βάρδια.
--	--

17. Πίνακας PrescriptionEvent

Περιορισμός / SQL	Σκεπτικό
UNIQUE(DoctorAMKA, HospitalizationID, DrugID, PrescriptionDate)	Υλοποιεί αμέσως το business rule του νοσοκομείου ότι κάθε συνδυασμός (ιατρός, ασθενής, φάρμακο, ημερομηνία έναρξης) είναι μοναδικός. Ο ίδιος γιατρός μπορεί να ξανα συνταγογραφήσει το ίδιο φάρμακο σε διαφορετική ημερομηνία για τον ίδιο ασθενή.
EndDate IS NULL OR EndDate >= StartDate	Η λήξη αγωγής δεν μπορεί να προηγείται της έναρξης. Επιτρέπει NULL για ανοιχτές (χωρίς ορισμένη λήξη) συνταγές.

18. Πίνακας Image

Ο πίνακας Image αντιστοιχίζει μία εικόνα σε μέλος προσωπικού, τμήμα, κλίνη ή αίθουσα επεμβάσεων. Ο παρακάτω περιορισμός εξασφαλίζει τη συνέπεια του σύνθετου composite key για κλίνες.

Περιορισμός / SQL	Σκεπτικό
chk_room_association: (RoomID IS NULL AND RoomDepartmentID IS NULL) OR (RoomID IS NOT NULL AND RoomDepartmentID IS NOT NULL)	Το composite FK της κλίνης απαιτεί και τα δύο στοιχεία (RoomID και DepartmentID) ταυτόχρονα. Δεν είναι εφικτό να οριστεί μόνο το ένα εκ των δύο — αυτό θα οδηγούσε σε πρόβλημα στο integrity της βάσης.

A.2.3.2 Triggers

Τα triggers χρησιμοποιούνται αποκλειστικά για κανόνες που απαιτούν ανάγνωση δεδομένων από άλλες γραμμές ή πίνακες, κάτι που δεν είναι εφικτό με απλούς CHECK ή UNIQUE περιορισμούς. Το install.sql περιέχει 21 triggers, οργανωμένα σε ομάδες. Κάθε trigger υλοποιείται και σε BEFORE INSERT και σε BEFORE UPDATE, ώστε οι κανόνες να ισχύουν ανεξαρτήτως της λειτουργίας που εκτελεί η εφαρμογή.

1. Ιεραρχία Εποπτείας Ιατρών

trg_doctor_before_insert, trg_doctor_before_update

Η ιεραρχία εποπτείας στη βάση δεδομένων αποτυπώνεται με ένα recursive self-join στον πίνακα Doctor. Ο κλάδος αυτός αποτελεί βασικό κανόνα, καθώς απαιτεί έλεγχο ολόκληρης της αλυσίδας εποπτείας για την αποτροπή κυκλικής εξάρτησης.

Κανόνας	Ανάλυση
Ειδικευόμενος → επόπτης υποχρεωτικός	Εάν Rank = 'Resident' AND SupervisorAMKA IS NULL → SIGNAL, (οι ειδικευόμενοι έχουν υποχρεωτικά επόπτη).
Διευθυντής → χωρίς επόπτη	Εάν Rank = 'Director' AND SupervisorAMKA IS NOT NULL → SIGNAL. Αντίστροφος κανόνας: η κορυφή της ιεραρχίας δεν έχει επόπτη.
Αυτο-εποπτεία	Εάν SupervisorAMKA = AMKA → SIGNAL. Ένας ιατρός δεν μπορεί να επιβλέπει τον εαυτό του.
Κυκλική εποπτεία	Χρησιμοποιεί Recursive CTE για να ανιχνεύσει αν τον AMKA του νέου ιατρού εμφανίζεται στην αλυσίδα των supervisor. Εάν ναι, η εισαγωγή απορρίπτεται. Ο συγκεκριμένος έλεγχος δεν μπορεί να εκφραστεί με κανένα static constraint.

2. Αλλεργίες και Ασφάλεια Συνταγογράφησης

trg_patient_allergy_check_before_insert, trg_patient_allergy_check_before_update

Αυτά τα triggers αποτελούν βασικό κανόνα για την ασφάλεια του ασθενή. Κάθε φορά που επιχειρείται δημιουργία ή τροποποίηση συνταγής, το trigger:

- Ανακτά τον ασθενή της νοσηλείας από το HospitalizationID της συνταγής.
- Συνενώνει (JOIN) τις δραστικές ουσίες του φαρμάκου (HasSubstances) με τις αλλεργίες του ασθενή (allergic_to).
- Εάν υπάρχει έστω μία κοινή ουσία, σηματοδοτεί σφάλμα με SIGNAL SQLSTATE '45000'.

Ο έλεγχος δεν μπορεί να γίνει με CHECK constraint, γιατί απαιτεί JOIN μεταξύ τριών πινάκων. Το trigger εκτελείται πριν από κάθε εισαγωγή ή ενημέρωση, ώστε καμία παραβίαση να μην καταχωρηθεί ποτέ στη βάση.

3. Προγραμματισμός Ιατρικών Πράξεων — Χρονική Επικάλυψη

trg_Procedure_Overlap_Insert, trg_Procedure_Overlap_Update

Δύο επεμβάσεις απαγορεύεται να πραγματοποιούνται ταυτόχρονα στον ίδιο χώρο, ή ο ίδιος κύριος χειρουργός να συμμετέχει σε δύο ταυτόχρονες επεμβάσεις. Το UNIQUE constraint στο ProcedureEvent(ProcRoomID, DateTime) αντιμετωπίζει μόνο την ακριβώς ίδια ώρα έναρξης· το trigger εκτελεί έλεγχο για επικάλυψη λαμβάνοντας υπόψη και τη διάρκεια:

NewEndTime = NEW.DateTime + INTERVAL ProcDuration MINUTE

OverlapCount = COUNT(*) WHERE (same_room OR same_main_doc)

AND (NEW.DateTime < OldEnd) -- νέα αρχίζει πριν παλιά τελειώσει

AND (OldStart < NewEndTime) -- παλιά αρχίζει πριν νέα τελειώσει

Ο παραπάνω έλεγχος overlap εφαρμόζεται τόσο για τον χώρο (ProcRoomID) όσο και για τον γιατρό (MainDocAMKA). Επίσης, ελέγχεται αν ο κύριος χειρουργός είναι ενεργός γιατρός (IsActive).

4. Βάρδιες Γιατρών

trg_hasdoctor_shift_before_insert,

trg_hasDoctor_prevent_removing_last_senior

trg_hasdoctor_shift_before_update,

Οι βάρδιες ιατρών συγκεντρώνουν τους περισσότερους επιχειρησιακούς κανόνες της βάσης. Τα δύο κύρια triggers (INSERT/UPDATE) ελέγχουν σειριακά τέσσερις ανεξάρτητους κανόνες:

Κανόνας	Ανάλυση
Ενεργό μέλος	Ελέγχει IsActive = 1 στο Staff. Δεν επιτρέπεται εφημερία σε inactive υπάλληλο.
Max 15 βάρδιες/μήνα	COUNT(*) βαρδιών για τον ίδιο γιατρό στον ίδιο μήνα/χρόνο. Εάν $\geq 15 \rightarrow$ SIGNAL.
Minimum 8ωρη ανάπαυση	Υπολογίζει το απόλυτο χρονικό διάστημα μεταξύ της νέας βάρδιας και κάθε ήδη καταχωρημένης. Ο έλεγχος χρησιμοποιεί $ABS(TIMESTAMPDIFF(HOUR,...)) < 16$, αφού κάθε βάρδια διαρκεί 8 ώρες: 8h εργασία + 8h ανάπαυση = 16h ελάχιστο διάστημα από αρχή σε αρχή.
Max 3 νυχτερινές βάρδιες σερί	Αν η νέα βάρδια είναι νυχτερινή, μετράει πόσες νυχτερινές υπάρχουν στις τρεις αμέσως προηγούμενες ημέρες. Εάν = 3 $\rightarrow$ SIGNAL.

Senior εποπτεία	Εάν ο ιατρός είναι Resident, ελέγχει αν υπάρχει ήδη ένας τουλάχιστον Registrar/Consultant/Director στην ίδια βάρδια. Αποτρέπει βάρδια ειδικευόμενου χωρίς επόπτη.
-----------------	---

Ένα ιδιαίτερης σημασίας τρίτο trigger, `trg_hasDoctor_prevent_removing_last_senior`, ενεργοποιείται κατά τη διαγραφή εγγραφής από το `hasDoctor`. Εάν ο ιατρός που διαγράφεται είναι ο μοναδικός `senior` σε βάρδια που περιλαμβάνει Ειδικευόμενο, η διαγραφή απορρίπτεται.

5. Βάρδιες Νοσηλευτών

trg_Nurse_Shift_Rules_Insert, trg_Nurse_Shift_Rules_Update

Αντίστοιχο σύνολο κανόνων με τις βάρδιες γιατρών, με τις εξής διαφορές: μέγιστο 20 βάρδιες/μήνα (αντί 15) και απουσία ελέγχου `senior-Resident`, ο οποίος αφορά αποκλειστικά γιατρούς. Η λογική δωρης ανάπαυσης και 3 συνεχόμενων νυχτερινών εφαρμόζεται ταυτόσημα.

Στο UPDATE trigger: η εγγραφή που ενημερώνεται εξαιρείται ρητά από τους υπολογισμούς (`NOT (DepartmentID = OLD.DepartmentID AND ...)`) ώστε να μην μετράει η ίδια εγγραφή κατά τον έλεγχο ορίων.

6. Βάρδιες Διοικητικού Προσωπικού

trg_Admin_Shift_Rules_Insert, trg_Admin_Shift_Rules_Update

Ίδια δομή με το 6, με μέγιστο 25 βάρδιες/μήνα για το διοικητικό προσωπικό, σύμφωνα με την εκφώνηση. Οι κανόνες ανάπαυσης και νυχτερινών βαρδιών ισχύουν ίδια και εδώ.

7. Αντιστοιχία Ασθενούς Triage–Νοσηλείας

trg_triage_hospitalization_same_patient_insert, trg_triage_hospitalization_same_patient_update

Κάθε φορά που ένα `TriageEvent` συνδέεται με `Hospitalization` (εισαγωγή μέσω αποδοχής), το trigger επαληθεύει ότι ο `PatientAMKA` του `TriageEvent` ταυτίζεται με τον `PatientAMKA` της αντίστοιχης νοσηλείας. Αν και η stored procedure `ProcessTriageAdmission` δημιουργεί αυτόματα και τις δύο εγγραφές με συνέπεια, το trigger αποτελεί δεύτερη γραμμή ελέγχου για απευθείας SQL εισαγωγές που παρακάμπτουν τη διαδικασία.

Το UPDATE trigger εφαρμόζει επιπλέον βελτιστοποίηση: εκτελεί τον έλεγχο μόνο αν έχει αλλάξει πράγματι το `HospitalizationID` ή το `PatientAMKA`, αξιοποιώντας τον NULL-safe operator για σύγκριση null τιμών.

8. Exclusive-Or για Αναφορά Εικόνας

trg_image_exclusive_insert, trg_image_exclusive_update

Στον πίνακα Image κάθε εικόνα αφορά ακριβώς έναν στόχο από τους τέσσερις δυνατούς (Staff, Department, Room, ProcedureRoom). Το CHECK constraint διαχειρίζεται μόνο την εσωτερική συνέπεια του σύνθετου composite key για κλίνες: ο πλήρης αμοιβαία αποκλειστικός έλεγχος δεν μπορεί να εκφραστεί με CHECK, καθώς απαιτεί μέτρηση nullable στηλών.

Το trigger υπολογίζει το πλήθος των non-null στηλών-στόχων:

```
cnt = (ProcRoomId IS NOT NULL) + (StaffAMK IS NOT NULL) + (DepartmentID IS NOT NULL) + (RoomID IS NOT NULL)
```

```
IF cnt != 1 THEN SIGNAL -- ακριβώς ένας στόχος επιτρέπεται
```

Η λογική αυτή εξασφαλίζει ότι καμία εικόνα δεν μπορεί να μην αναφέρεται πουθενά (cnt = 0) ή να συνδέεται με παραπάνω από ένα (cnt > 1).

9. Evaluation Security (2 Level - Για γιατρούς και νοσηλεία)

trg_hosp_eval_insert, trg_doc_eval_insert

Τα συγκεκριμένα triggers υλοποιούν την αυστηρή επιχειρησιακή λογική (business logic) που διέπει τη διαδικασία αξιολόγησης του νοσοκομείου και του ιατρικού προσωπικού από τους ασθενείς.

trg_hosp_eval_insert (Αξιολόγηση Νοσηλείας): Το trigger αυτό προστατεύει τον πίνακα HospEvaluation. Πριν από κάθε εισαγωγή νέας αξιολόγησης (BEFORE INSERT), ελέγχει το πεδίο ExitDateTime της αντίστοιχης νοσηλείας στον πίνακα Hospitalization. Εάν η ημερομηνία εξόδου είναι NULL (δηλαδή η νοσηλεία βρίσκεται ακόμη σε εξέλιξη), η συναλλαγή απορρίπτεται άμεσα μέσω της εντολής SIGNAL SQLSTATE. Αυτό διασφαλίζει προγραμματιστικά τον χρονικό περιορισμό ότι η αξιολόγηση επιτρέπεται αυστηρά και μόνο μετά την ολοκλήρωση της νοσηλείας και την έξοδο του ασθενούς.

trg_doc_eval_insert (Αξιολόγηση Ιατρών): Το συγκεκριμένο trigger εφαρμόζει έναν πιο σύνθετο, διπλό έλεγχο ακεραιότητας για την προστασία του πίνακα DoctorEvaluation:

1. Χρονικός Έλεγχος : Όπως και στην αξιολόγηση των εγκαταστάσεων, επιβεβαιώνει αρχικά ότι η νοσηλεία έχει ολοκληρωθεί επιτυχώς, ελέγχοντας το ExitDateTime.
2. Έλεγχος Δικαιώματος Αξιολόγησης : Στη συνέχεια, υλοποιεί τον αυστηρό περιορισμό των απαιτήσεων του συστήματος, ο οποίος ορίζει ότι ο ασθενής μπορεί να αξιολογήσει μόνο τους ιατρούς που του συνταγογράφησαν φάρμακα κατά τη διάρκεια της νοσηλείας του. Το σύστημα εκτελεί μια δυναμική συνάθροιση (SELECT COUNT(*)) στον πίνακα PrescriptionEvent, αναζητώντας τον συγκεκριμένο συνδυασμό HospitalizationID και DoctorAMKA. Εάν το αποτέλεσμα είναι μηδέν (v_PrescriptionCount = 0), η εισαγωγή

ακυρώνεται. Αυτό αποτρέπει μαθηματικά την καταχώρηση παραπλανητικών αξιολογήσεων για γιατρούς που δεν ενεπλάκησαν ποτέ στη φαρμακευτική αγωγή του συγκεκριμένου ασθενούς.

A.2.3.3 Στρατηγική σε DELETE - UPDATE

Η διαχείριση του referential integrity αποτελεί μία από τις πιο βασικές σχεδιαστικές αποφάσεις ενός Relational schema, καθώς καθορίζει τη συμπεριφορά της βάσης σε κάθε αλλαγή ή διαγραφή δεδομένων. Για κάθε Foreign Key επιλέχθηκε συνειδητά μία από τις διαθέσιμες πολιτικές CASCADE, RESTRICT ή η απουσία ενέργειας (default RESTRICT), με γνώμονα τη σχέση ιδιοκτησίας μεταξύ πίνακα parent και child, τον ρόλο των δεδομένων στο σύστημα και την ανάγκη προστασίας του ιατρικού αρχείου και του ιστορικού του νοσοκομείου. Στη συνέχεια αναλύεται η λογική κάθε επιλογής, ομαδοποιημένη ανά στρατηγική.

Η λογική που διέπει όλες τις επιλογές referential integrity στο σχήμα βασίζεται σε μία κεντρική ερώτηση: τι συμβαίνει στα δεδομένα του παιδιού όταν ο γονέας αλλάζει ή φεύγει;

Ορίστηκαν τρεις διακριτές στρατηγικές. Τα δεδομένα που δεν έχουν ανεξάρτητο κύκλο ζωής τηλέφωνα, αντιστοιχίσεις ιατρού-τιμήματος, σύνθεση ομάδων χειρουργείου, εφημερίες, ουσίες φαρμάκων χρησιμοποιούν CASCADE και στις δύο κατευθύνσεις: αν ο γονέας διαγραφεί, τα παιδιά δεν έχουν λόγο ύπαρξης και φεύγουν μαζί του.

Για τα physical keys προσωπικού και ασθενών (ΑΜΚΑ) η λογική διαχωρίζεται: διαγραφή RESTRICT, ενημέρωση CASCADE. Ένας γιατρός με κλινικό ιστορικό δεν μπορεί να διαγραφεί, αλλά αν ο κωδικός του διορθωθεί μέσω stored procedure, η αλλαγή διαδίδεται αυτόματα σε κάθε πίνακα που τον αναφέρει. Αυτό είναι ακριβώς η δουλειά που κάνουν οι UpdateDoctor/Nurse/AdminStaff procedures με ένα μόνο UPDATE στο Staff.

Το κλινικό αρχείο — νοσηλείες, triage, αξιολογήσεις, πράξεις, συνταγές — χρησιμοποιεί RESTRICT και σε delete και update. Τα κλειδιά εδώ είναι surrogate integers που δεν αλλάζουν ποτέ, οπότε ON UPDATE δεν χρειάζεται. Το ON DELETE RESTRICT εξασφαλίζει ότι το ιστορικό παραμένει ακέραιο: δεν μπορεί να διαγραφεί κλίνη που έχει νοσηλείες, ούτε νοσηλεία που έχει αξιολόγηση ή συνταγή.

Εν κατακλείδι, το σχήμα διαχωρίζει ανάμεσα σε δεδομένα που ανήκουν σε κάτι, δεδομένα που αναφέρονται σε κάτι με δυνατότητα διόρθωσης, και δεδομένα που αποτελούν αρχείο και δεν πρέπει να χαθούν ποτέ.

A.2.4. Stored Procedures

Οι stored procedures χρησιμοποιούνται για σύνθετες λειτουργίες, περιέχοντας transactions που πρέπει να είναι ατομικά. Υλοποιούνται με δομές START TRANSACTION, COMMIT και exit handler για ROLLBACK, ώστε η βάση να μη μένει σε ενδιάμεση κατάσταση αν αποτύχει μέρος της πράξης.

Αναλυτική παρουσίαση των Stored Procedures που χρειάστηκαν:

➤ RegisterDoctor, RegisterNurse, RegisterAdminStaff :

Οι συγκεκριμένες αποθηκευμένες διαδικασίες αναλαμβάνουν την ασφαλή εγγραφή νέου προσωπικού στο σύστημα. Η λογική τους λειτουργεί σε δύο επίπεδα για να ικανοποιήσει την ιεραρχία (superclass/subclass) του σχεσιακού μας μοντέλου.

Πιο συγκεκριμένα, η διαδικασίες εφαρμόζουν ελέγχους εγκυρότητας, όπως τον έλεγχο για την ημερομηνία πρόσληψης ('HireDate'), η οποία αν υπερβαίνει την τρέχουσα ημερομηνία προκαλεί σκόπιμη διακοπή μέσω της εντολής **SIGNAL SQLSTATE**. Η αρχιτεκτονική της εισαγωγής εκτελείται σε δύο φάσεις:

- **Εισαγωγή στην Υπερκλάση:** Τα γενικά στοιχεία εισάγονται στον υπερ-πίνακα *Staff*. Στο σημείο αυτό, το πεδίο 'Type' ορίζεται ρητά (π.χ. 'Doctor' ή 'Nurse') προκειμένου να επιβληθεί προγραμματιστικά ο disjointness constraint, αποτρέποντας έναν υπάλληλο από το να ανήκει σε πολλαπλούς ασύμβατους τύπους ταυτόχρονα.
- **Εισαγωγή στην Υποκλάση:** Αμέσως μετά, τα εξειδικευμένα χαρακτηριστικά (όπως η 'Specialty' ή το 'Rank') εισάγονται στον αντίστοιχο υπο-πίνακα (π.χ. Doctor), χρησιμοποιώντας το ίδιο ακριβώς πρωτεύον κλειδί ('AMKA').

➤ UpdateDoctorInfo, UpdateNurseInfo, UpdateAdminStaffInfo:

Οι παραπάνω διαδικασίες ενημέρωσης επιτρέπουν την τροποποίηση των χαρακτηριστικών του προσωπικού, συμπεριλαμβανομένου και του ίδιου του Πρωτεύοντος Κλειδιού ('AMKA'). Για να επιτευχθεί αυτό με ασφάλεια, η διαδικασία αξιοποιεί τους περιορισμούς αναφορικής ακεραιότητας (Referential Integrity) του DBMS.

- **Ενημέρωση Υπερκλάσης με Cascade:** Η ενημέρωση ξεκινά από τον πίνακα *Staff*. Προσθέτοντας τον έλεγχο AND Type = 'Doctor', η βάση εγγυάται ότι δεν θα τροποποιηθεί κατά λάθος εγγραφή άλλης ειδικότητας. Όταν το σύστημα ενημερώνει το 'AMKA' στον υπερ-πίνακα *Staff*, ο μηχανισμός **ON UPDATE CASCADE**, που έχει οριστεί κατά τη δημιουργία του σχήματος, προωθεί αυτόματα τη νέα τιμή του κλειδιού στον υπο-πίνακα *Doctor* και σε όλους τους συσχετιζόμενους πίνακες (π.χ. εφημερίες).
- **Ενημέρωση Υποκλάσης:** Στο δεύτερο βήμα, η διαδικασία ενημερώνει τα εξειδικευμένα πεδία (όπως 'License', 'Specialty') απευθείας στον πίνακα της υποκλάσης. Εδώ,

χρησιμοποιείται το νέο AMKA (p_NewAMKA), καθώς ο μηχανισμός cascade έχει ήδη ενημερώσει το ξένο κλειδί στο παρασκήνιο.

➤ **CreateDepartmentWithDirector & UpdateDepartmentWithDirector:**

Οι συγκεκριμένες αποθηκευμένες διαδικασίες διαχειρίζονται τον κύκλο ζωής των τμημάτων του νοσοκομείου. Στο τέλος της συναλλαγής, το σύστημα εγγυάται ότι η βάση δεδομένων θα παραμείνει σε μια απολύτως έγκυρη και συνεπή κατάσταση που ικανοποιεί όλους τους περιορισμούς του σχεσιακού σχήματος.

Διαδικασία CreateDepartmentWithDirector() : Η παρούσα διαδικασία αναλαμβάνει την ταυτόχρονη δημιουργία ενός νέου τμήματος και την ανάθεση του Διευθυντή του.

- **Εισαγωγή και Ανάκτηση Κλειδιού:** Αρχικά, η διαδικασία εισάγει τα στοιχεία του νέου τμήματος στον πίνακα Department. Επειδή το 'DepartmentID' παράγεται αυτόματα (AUTO_INCREMENT), η διαδικασία αξιοποιεί τη συνάρτηση **LAST_INSERT_ID()** για να ανακτήσει με ασφάλεια το παραγόμενο πρωτεύον κλειδί στη μεταβλητή v_DeptID.
- **Αναφορική Συσχέτιση:** Αμέσως μετά, εισάγει τη συσχέτιση του Διευθυντή (p_DirectorAMKA) με το νέο τμήμα (v_DeptID) στον πίνακα συσχέτισης DoctorDepartment. Αν οποιοδήποτε βήμα αποτύχει, το **ROLLBACK** διασφαλίζει ότι δεν θα δημιουργηθεί ποτέ ένα τμήμα χωρίς σωστά καταχωρημένο διευθυντή.

Διαδικασία UpdateDepartmentWithDirector(): Η διαδικασία αυτή διαχειρίζεται την αλλαγή του Διευθυντή σε ένα ήδη υφιστάμενο τμήμα, διασφαλίζοντας την ακεραιότητα των δεδομένων κατά τη μετάβαση.

- **Ασφαλής Εισαγωγή στον Πίνακα Συσχέτισης (Conditional Insert):** Πριν γίνει η ανάθεση της διεύθυνσης, η διαδικασία χρησιμοποιεί τη δομή ελέγχου **IF NOT EXISTS**. Με αυτόν τον τρόπο ελέγχει δυναμικά εάν ο νέος διευθυντής ανήκει ήδη στο τμήμα. Αν δεν ανήκει, τον εισάγει αυτόματα στον πίνακα DoctorDepartment. Αυτή η λογική αποτρέπει σφάλματα παραβίασης διπλότυπων κλειδιών στην περίπτωση που ο ιατρός εργαζόταν ήδη στο τμήμα.
- **Ενημέρωση Τμήματος:** Τέλος, το πεδίο 'DirectorAMKA' του πίνακα Department ενημερώνεται με το 'AMK' του νέου διευθυντή και η συναλλαγή γίνεται COMMIT.

➤ **AdmitPatient:**

Η συγκεκριμένη αποθηκευμένη διαδικασία διαχειρίζεται το επιχειρησιακό σενάριο της εισαγωγής ενός ασθενούς στο νοσοκομείο, χωρίς να περάσει από triage, δηλαδή να έχει κλείσει ραντεβού.

Η αρχιτεκτονική της εκτέλεσης χωρίζεται σε τρεις φάσεις:

- **Έλεγχος Εγκυρότητας:** Αρχικά, το σύστημα ελέγχει αν το προφίλ του ασθενούς είναι ενεργό (isActive = 1). Εάν δεν είναι, η εκτέλεση διακόπτεται σκόπιμα μέσω της εντολής **SIGNAL SQLSTATE**, επιστρέφοντας το αντίστοιχο μήνυμα σφάλματος.
- **Αποτροπή Ταυτόχρονης Πρόσβασης:** Σε ένα περιβάλλον πολλαπλών χρηστών, δύο διαχειριστές θα μπορούσαν να προσπαθήσουν να δεσμεύσουν το ίδιο δωμάτιο ταυτόχρονα. Για να αποτραπεί αυτό, η διαδικασία διαβάζει την κατάσταση του δωματίου χρησιμοποιώντας τον όρο **FOR UPDATE**. Αυτό επιβάλλει ένα αποκλειστικό κλείδωμα στη συγκεκριμένη γραμμή του πίνακα Room. Εάν μια άλλη συναλλαγή προσπαθήσει να ελέγξει το ίδιο δωμάτιο, αναγκάζεται να περιμένει στην ουρά μέχρι να ολοκληρωθεί η τρέχουσα συναλλαγή.
- **Ενημέρωση Κατάστασης:** Εφόσον το δωμάτιο βρεθεί διαθέσιμο ('Available'), η διαδικασία αλλάζει την κατάστασή του σε 'Occupied' και καταχωρεί την επίσημη εγγραφή της νοσηλείας στον πίνακα Hospitalization, ολοκληρώνοντας επιτυχώς τη συναλλαγή με **COMMIT**.

➤ **DischargePatient:**

Η συγκεκριμένη αποθηκευμένη διαδικασία υλοποιεί την επιχειρησιακή λογική του εξιτηρίου ενός ασθενούς, λειτουργώντας ως ο άμεσος μηχανισμός αποδέσμευσης πόρων του συστήματος.

Η αρχιτεκτονική της εκτέλεσης ακολουθεί μια ροή τριών βημάτων:

- **Ανάκτηση Πληροφορίας Κλίνης :** Αρχικά, η διαδικασία εκτελεί ένα **SELECT ... INTO** για να ανακτήσει το συγκεκριμένο RoomID και DepartmentID που είχαν δεσμευτεί κατά τη νοσηλεία (HospitalizationID). Τα δεδομένα αυτά αποθηκεύονται σε τοπικές μεταβλητές.
- **Καταγραφή Εξόδου:** Στη συνέχεια, ενημερώνει (**UPDATE**) την εγγραφή της νοσηλείας στον πίνακα Hospitalization, καταγράφοντας το ακριβές χρονικό στίγμα αναχώρησης ('ExitDateTime'). Αξίζει να σημειωθεί ότι αυτή η ενέργεια είναι κρίσιμη για το σχήμα μας, καθώς η εισαγωγή αυτής της ημερομηνίας ενεργοποιεί αυτόματα τον υπολογισμό του παραγόμενου (derived) πεδίου 'ActualDays' (που έχει επιλεγεί να δημιουργηθεί ως **generated virtual column**), το οποίο με τη σειρά του απαιτείται για την τιμολόγηση.
- **Απελευθέρωση Πόρων (Capacity Reallocation):** Στο τελικό βήμα, ενημερώνει την κατάσταση του αντίστοιχου δωματίου στον πίνακα Room, μεταβάλλοντας το πεδίο 'State' από 'Occupied' ξανά σε 'Available'. Η ατομικότητα της συναλλαγής εγγυάται ότι ένα δωμάτιο δεν θα εμφανιστεί ποτέ ως διαθέσιμο εάν το σύστημα αποτύχει να καταγράψει την ώρα εξόδου του ασθενούς, προστατεύοντας τη βάση από λογικά σφάλματα διαθεσιμότητας κλινών .

➤ **CalculateHospitalizationBill:**

Η αποθηκευμένη διαδικασία ενσωματώνει την πολύπλοκη επιχειρησιακή λογική της τιμολόγησης μιας νοσηλείας, υλοποιώντας το σύστημα των Κλειστών Ενοποιημένων Νοσηλίων (Κ.Ε.Ν.) του νοσοκομείου. Επειδή η οικονομική εκκαθάριση απαιτεί τον συνδυασμό ετερογενών δεδομένων (ημέρες νοσηλείας, φάρμακα, εργαστηριακές εξετάσεις, χειρουργεία), η διαδικασία εξάγει τα

αποτελέσματα της μέσω παραμέτρων εξόδου (OUT), προσφέροντας ένα ασφαλές και αρθρωτό API στο επίπεδο της εφαρμογής.

Η αρχιτεκτονική της εκτέλεσης ακολουθεί τέσσερα αυστηρά υπολογιστικά βήματα:

- **Ανάκτηση Βασικού Κόστους & Ημερών:** Μέσω λειτουργίας JOIN μεταξύ των πινάκων Hospitalization και Cost, ανακτάται το βασικό κόστος ('BaseCost'), η προβλεπόμενη Μέση Διάρκεια Νοσηλείας (PredictedAvgTime) και η πραγματική διάρκεια παραμονής του ασθενούς (ActualDays).
- **Υπολογισμός Πρόσθετων Τελών (Additional Fees):** Η διαδικασία ελέγχει δυναμικά εάν η πραγματική παραμονή υπερβαίνει την προβλεπόμενη ΜΔΝ. Εάν ισχύει, επιβάλλει αναλογική ημερήσια χρέωση για τις επιπλέον ημέρες μέσω του μαθηματικού τύπου $(ActualDays - PredictedDays) * ChargePerDay$. Διαφορετικά, η πρόσθετη χρέωση μηδενίζεται.
- **Συνάθροιση Εργαστηριακών και Επεμβατικών Κόστων (Cost Aggregation):** Στη συνέχεια, η διαδικασία αθροίζει τα επιμέρους κόστη από τις εργαστηριακές εξετάσεις (HospLabTest, LabTest) και τις ιατρικές πράξεις (ProcedureEvent, ProcedureType) του ασθενούς. Στο σημείο αυτό, γίνεται χρήση της συνάρτησης `IFNULL(SUM(...), 0)`. Στη σχεσιακή άλγεβρα, εάν η συνάρτηση συνάθροισης δεν βρει εγγραφές (π.χ. ο ασθενής δεν έκανε καμία εξέταση), επιστρέφει NULL, το οποίο προκαλεί αποτυχία σε οποιαδήποτε επακόλουθη μαθηματική πράξη. Με την IFNULL, το σύστημα προστατεύεται, μετατρέποντας τα κενά σύνολα στο ακέραιο 0.
- **Τελική Εκκαθάριση:** Το άθροισμα του Βασικού Κόστους, των Πρόσθετων Τελών, των Εργαστηριακών και των Επεμβατικών εξόδων υπολογίζεται και αποθηκεύεται με ασφάλεια στην παράμετρο εξόδου `p_TotalFees`.

➤ **FetchNext:**

Η παρούσα αποθηκευμένη διαδικασία υλοποιεί την διαδικασία “pop” από την ουρά προτεραιότητας του Τμήματος Επειγόντων Περιστατικών (Triage).

Λογική:

- **Φιλτράρισμα Κατάστασης:** Αρχικά, η διαδικασία φιλτράρει τις εγγραφές του πίνακα TriageEvent, απομονώνοντας αποκλειστικά τα περιστατικά που βρίσκονται σε κατάσταση / ουρά αναμονής προς εξέταση (Outcome = 'Pending').
- **Συνένωση Δεδομένων :** Μέσω της πράξης JOIN με τον πίνακα Patient, το σύστημα συνδέει το 'ΑΜΚΑ' του ασθενούς για να ανακτήσει τα προσωπικά του στοιχεία (όνομα και επώνυμο), προσφέροντας άμεση πληροφορία στο ιατρικό προσωπικό.
- **Περιορισμός Αποτελέσματος (Limiting):** Τέλος, παίρνει από την ουρά αναμονής τον πρώτο σε προτεραιότητα ασθενή που περιμένει, με LIMIT 1.

➤ ProcessTriageAdmission:

Η παρούσα αποθηκευμένη διαδικασία υλοποιεί την μετάβαση ενός ασθενούς από το Τμήμα Επειγόντων Περιστατικών (Triage Queue) σε καθεστώς επίσημης νοσηλείας.

Η αρχιτεκτονική της εκτέλεσης ακολουθεί μια ροή τεσσάρων φάσεων:

- **Διπλό Κλείδωμα για Έλεγχο Ταυτόχρονης Πρόσβασης:** Το σύστημα επιβάλλει δύο διαδοχικά αποκλειστικά κλειδώματα αξιοποιώντας τον όρο FOR UPDATE. Αρχικά, κλειδώνει τη συγκεκριμένη εγγραφή του TriageEvent ώστε να αποτραπεί η ταυτόχρονη εξέταση του ίδιου περιστατικού από δεύτερο ιατρό. Στη συνέχεια, διαβάζει και κλειδώνει τη ζητούμενη κλίνη (Room). Αν η κλίνη δεν είναι διαθέσιμη, η διαδικασία εκτελεί ελεγχόμενη διακοπή (SIGNAL SQLSTATE).
- **Ενημέρωση Χωρητικότητας :** Εφόσον οι πόροι έχουν δεσμευτεί με ασφάλεια, η διαδικασία μεταβάλλει την κατάσταση του δωματίου ('State') σε 'Occupied', αποτρέποντας μελλοντικές κρατήσεις.
- **Εισαγωγή Νοσηλείας & Ανάκτηση Κλειδιού :** Το σύστημα καταχωρεί τη νέα νοσηλεία στον πίνακα Hospitalization. Αμέσως μετά, αξιοποιεί τη συνάρτηση LAST_INSERT_ID() για να ανακτήσει το παραγόμενο πρωτεύον κλειδί (HospitalizationID) και να το αποθηκεύσει στην τοπική μεταβλητή v_NewHospitalizationID.
- **Ολοκλήρωση Διαλογής & Αναφορική Συσχέτιση:** Στο τελικό βήμα, η διαδικασία επιστρέφει στην εγγραφή του TriageEvent. Ενημερώνει το Outcome σε 'Accepted', καταγράφει τον χρόνο ολοκλήρωσης της ιατρικής αξιολόγησης (AssessmentDateTime), και εισάγει το v_NewHospitalizationID στο αντίστοιχο ξένο κλειδί. Με τον τρόπο αυτό, συνδέει το ιστορικό των επειγόντων με τον φάκελο νοσηλείας πριν εκτελέσει το τελικό COMMIT.

➤ DiscardTriagePatient:

Η τελική αποθηκευμένη διαδικασία του συστήματος διαχειρίζεται το σενάριο όπου ο ιατρός, κατόπιν της κλινικής αξιολόγησης στο Τμήμα Επειγόντων Περιστατικών κρίνει ότι ο ασθενής δεν χρήζει εισαγωγής για περαιτέρω νοσηλεία. Η διαδικασία αυτή ολοκληρώνει τον κύκλο ζωής του περιστατικού και εξασφαλίζει την ορθή καταγραφή των ιατρικών αποφάσεων.

Η αρχιτεκτονική της εκτέλεσης βασίζεται στα εξής δύο βήματα:

- **Ασφαλής Μετάβαση Κατάστασης:** Με τον περιορισμό AND Outcome = 'Pending' στο WHERE clause, εξασφαλίζουμε ότι η απόρριψη μπορεί να εφαρμοστεί αποκλειστικά και μόνο σε ενεργά περιστατικά. Εάν ένας άλλος ιατρός έχει ήδη κάνει αποδοχή (ProcessTriageAdmission) του ίδιου ασθενούς την ίδια ακριβώς στιγμή, η τρέχουσα εντολή θα αγνοήσει την εγγραφή, αποτρέποντας τη λογική παραβίαση των καταστάσεων (state corruption).

- **Timestamping & Ενημέρωση 'Outcome'** : Εφόσον η συνθήκη ικανοποιηθεί, το πεδίο 'Outcome' ενημερώνεται οριστικά σε 'Discarded' και καταγράφεται το ακριβές χρονικό στίγμα ολοκλήρωσης της εξέτασης ('AssessmentDateTime').

A.2.5. Ορισμός Ευρετηρίων (Indexing)

Η δημιουργία ευρετηρίων (indexes) αποτελεί τον ακρογωνιαίο λίθο της φυσικής βελτιστοποίησης (physical optimization) μιας βάσης δεδομένων. Χωρίς την ύπαρξη κατάλληλων ευρετηρίων, το Σύστημα Διαχείρισης (DBMS) αναγκάζεται να εκτελεί πλήρεις σαρώσεις πινάκων (Full Table Scans), διαδικασία με γραμμικό κόστος $O(N)$ που καθίσταται απαγορευτική καθώς αυξάνεται ο όγκος των δεδομένων. Αντίθετα, η χρήση δεικτών επιτρέπει στο σύστημα να προσπελάσει τα δεδομένα μέσω δομών δέντρου με λογαριθμικό κόστος προσπέλασης $O(\log N)$.

Στη μηχανή InnoDB της MySQL, τα ευρετήρια υλοποιούνται κατά κανόνα ως δομές δέντρων B+-Trees. Είναι σημαντικό να τονιστεί ότι η MySQL δημιουργεί αυτόματα clustered indexes για όλα τα Primary Keys, καθώς και δευτερεύοντα ευρετήρια (secondary indexes) για τα Foreign Keys και για τα πεδία που υπόκεινται σε περιορισμό Μοναδικότητας (UNIQUE constraints). Επομένως, οι έλεγχοι ακεραιότητας, καθώς και τα συχνά JOINS μεταξύ των κεντρικών πινάκων του σχήματός μας (π.χ. Hospitalization με Patient μέσω του PatientAMKA), υποστηρίζονται ήδη στο βέλτιστο επίπεδο από το ίδιο το Σύστημα Διαχείρισης, χωρίς να απαιτείται η χειροκίνητη δημιουργία επιπλέον δεικτών για αυτά τα πεδία.

Η στρατηγική δημιουργίας των δευτερευόντων ευρετηρίων (secondary indexes) βασίστηκε στην ανάλυση των συχνότερων μοτίβων προσπέλασης (access patterns) και των λειτουργικών αναγκών ενός πραγματικού Πληροφοριακού Συστήματος Νοσοκομείου (όπως, για παράδειγμα, η συχνή αναζήτηση ιατρών ανά ειδικότητα ή το φιλτράρισμα νοσηλειών ανά ημερομηνία). Παράλληλα, αυτά τα γενικής χρήσης ευρετήρια σχεδιάστηκαν έτσι ώστε να εξυπηρετούν στο μέγιστο βαθμό και την εκτέλεση των 15 ζητούμενων ερωτημάτων (queries) της εργασίας, επιταχύνοντας δραματικά την αξιολόγηση σύνθετων φίλτρων (WHERE), ομαδοποιήσεων (GROUP BY) και ταξινομήσεων (ORDER BY). Πιο συγκεκριμένα, υλοποιήθηκαν τα ακόλουθα στοχευμένα ευρετήρια:

1. Ευρετήρια Ονομαστικής Αναζήτησης (Ασθενείς & Προσωπικό)

```
CREATE INDEX idx_staff_name ON Staff(LastName, FirstName);
```

```
CREATE INDEX idx_patient_name ON Patient(LastName, FirstName);
```

Σε ρεαλιστικές συνθήκες, η συχνότερη αναζήτηση στη γραμματεία ή στο τμήμα HR δεν γίνεται με το AMKA, αλλά με ονομαστικά στοιχεία. Τεχνικά, τα πεδία αυτά διαθέτουν πολύ υψηλό cardinality (high distinct values), επιτρέποντας στον optimizer να απομονώσει ακαριαία τις εγγραφές (index seek). Αξιοποιώντας τον Κανόνα Αριστερού Προθέματος (Leftmost Prefix Rule) των B+-Trees στη MySQL, τα παραπάνω composite indexes καλύπτουν ταυτόχρονα ερωτήματα που φιλτράρουν είτε μόνο με το LastName, είτε με τον συνδυασμό LastName AND FirstName.

2. Ευρετήρια Δημογραφικών Στοιχείων και Ειδικότητας (Equality & Range Lookups)

```
CREATE INDEX idx_doc_specialty ON Doctor(Specialty);
```

```
CREATE INDEX idx_age ON Staff(BirthDate);
```

```
CREATE INDEX idx_hasdoc_shiftdate ON hasDoctor(ShiftDate);
```

```
CREATE INDEX idx_hasnurse_shiftdate ON hasNurse(ShiftDate);
```

```
CREATE INDEX idx_hasadmin_shiftdate ON hasAdmin(ShiftDate);
```

Τα παραπάνω δευτερεύοντα ευρετήρια (secondary indexes) βελτιστοποιούν τα διαρκή ερωτήματα εύρεσης ιατρών, προγραμματισμού βαρδιών και παραγωγής δημογραφικών αναφορών. Τεχνικά η επιλογή τους δικαιολογείται ως εξής:

- Βελτιστοποίηση Ισότητας (Point Queries): Το idx_doc_specialty επιταχύνει τα ερωτήματα ακριβούς αντιστοίχισης (equality lookups). Ο Query Optimizer αποφεύγει το Full Table Scan (type: ALL στο EXPLAIN) και εκτελεί στοχευμένη προσπέλαση (type: ref) μέσω της δομής B+-Tree.
- Επιτάχυνση Range Scans: Στα ευρετήρια ημερομηνιών (BirthDate, ShiftDate), τα B+-Trees διατηρούν τα δεδομένα αυστηρά λογικά ταξινομημένα στους κόμβους-φύλλα. Σε ανισοτικές συγκρίσεις (<, >, BETWEEN), το σύστημα βρίσκει ακαριαία το όριο ($O(\log N)$) και εκτελεί γρήγορη σειριακή σάρωση (Index Range Scan, type: range).
 - * Σημείωση: Αν και τα range scans σε δευτερεύοντα ευρετήρια ενέχουν τον κίνδυνο χρονοβόρων τυχαίων προσπελάσεων στον δίσκο (Random I/Os) για την ανάκτηση των δεδομένων, ο Optimizer προστατεύει το σύστημα. Για στενά χρονικά παράθυρα εκτελεί το γρήγορο Index Scan, ενώ αν ζητηθεί τεράστιο διάστημα, αναγνωρίζει το I/O overhead και μεταβαίνει δυναμικά στο πιο αποδοτικό Full Table Scan.

3. Βελτιστοποίηση Ταξινομήσεων και Αποφυγή Filesort

```
CREATE INDEX idx_triage_queue ON TriageEvent(Outcome, EmergencyLevel, TriageDateTime);
```

Στο σύστημα του νοσοκομείου, ο πίνακας ελέγχου (dashboard) του ΤΕΠ αναζητά διαρκώς τον επόμενο ασθενή προς εξέταση. Η επιχειρησιακή αυτή λογική απαιτεί την απομόνωση των εκκρεμών περιστατικών (WHERE Outcome = 'Pending') και την επιστροφή τους σε αυστηρή σειρά προτεραιότητας βάσει επιπέδου επείγοντος και χρόνου άφιξης (ORDER BY EmergencyLevel ASC, TriageDateTime ASC).

Φιλτράρισμα Ισότητας (Equality Lookup): Βάσει του Κανόνα Αριστερού Προθέματος (Leftmost Prefix Rule), η τοποθέτηση του πεδίου Outcome στην πρώτη θέση επιτρέπει στον Optimizer να εκτελέσει ένα ακαριαίο index seek. Η μηχανή InnoDB απομονώνει κατευθείαν τον κλάδο του B+-Tree που περιέχει μόνο τα 'Pending' περιστατικά, αγνοώντας πλήρως όσα έχουν ήδη εξυπηρετηθεί ή απορριφθεί, χωρίς να ελέγχει τα δεδομένα (row-by-row filtering).

Πλήρης Αποφυγή Filesort: Εντός του κλάδου των 'Pending' περιστατικών, η δομή του B+-Tree διατηρεί τα δεδομένα ήδη προ-ταξινομημένα λεξικογραφικά (lexicographical ordering) ως προς τις επόμενες στήλες του δείκτη (EmergencyLevel και έπειτα TriageDateTime). Έτσι, το DBMS ανακτά τα δεδομένα με την ακριβή σειρά που ζητά το ORDER BY, εξαλείφοντας εντελώς την ανάγκη για τη χρονοβόρα δημιουργία in-memory πινάκων ταξινόμησης (αποτρέποντας τις λειτουργίες "Using filesort" και "Using temporary").

Μηδενισμός Random I/Os μέσω Top-N Αναζήτησης: Μολονότι τα δευτερεύοντα ευρετήρια επιβάλλουν τυχαίες προσπελάσεις (Random Disk Seeks) για την ανάκτηση ολόκληρων των εγγραφών, στη συγκεκριμένη περίπτωση η διαδικασία αναζητά μόνο τον επόμενο ασθενή (π.χ. μέσω LIMIT 1 στη Stored Procedure FetchNext). Ο δείκτης επιτρέπει στη βάση να εντοπίσει και να διαβάσει απλώς τον πρώτο διαθέσιμο κόμβο-φύλλο (first leaf node) του δέντρου και να τερματίσει την εκτέλεση ακαριαία, καθιστώντας την πολυπλοκότητα της ανάκτησης $O(\log N)$ ανεξαρτήτως του συνολικού όγκου δεδομένων. Σε κάθε περίπτωση όμως ακόμα και ολόκληρο το Triage Queue να απαιτηθεί, τα λίγα random I/O είναι σαφώς μικρότερα από το να γίνει sort (πιθανότατα με κάποιον αλγόριθμο όπως ο external sort) ολόκληρος ο πίνακας TriageEvent, που θα έχει ως εγγραφές και όλες τις ιστορικές αφίξεις των ασθενών.

4. Σύνθετα Ευρετήρια Κάλυψης (Covering Indexes)

CREATE INDEX idx_pat_days_in_hospital ON Hospitalization(PatientAMKA, AdmissionDateTime, ExitDateTime);

Η ανάκτηση του ιστορικού νοσηλειών ενός ασθενούς για συγκεκριμένο χρονικό παράθυρο είναι θεμελιώδης λειτουργία. Το σύνθετο ευρετήριο idx_pat_days_in_hospital δεν επιταχύνει απλώς το φιλτράρισμα, αλλά είναι ικανό να συμπεριφερθεί ως "Covering Index" για συγκεκριμένα queries αναφορών: εάν ένα ερώτημα επιλέγει μόνο αυτές τις τρεις στήλες (ή μαζί και το HospitalizationID, αφού λόγω της δομής των δευτερευόντων ευρετηρίων στην SQL αντί για δείκτες σε εγγραφές, αποθηκεύουν το primary key σαν τιμή), το DBMS απαντά εξ ολοκλήρου από τις σελίδες του ευρετηρίου (κατάσταση που εμφανίζεται ως **Using index στο πλάνο του EXPLAIN**), χωρίς να πραγματοποιήσει καθόλου random page fetches από τον κύριο πίνακα δεδομένων.

A.2.6. Views

Τα views προστέθηκαν για τρεις λόγους: να κρύβουν επαναλαμβανόμενα JOINS που εμφανίζονται σε πολλά queries, να παρέχουν έτοιμες φιλτραρισμένες όψεις ενεργών εγγραφών χωρίς να επαναλαμβάνεται ο έλεγχος IsActive σε κάθε query, και να υποστηρίζουν δύο λειτουργικές ανάγκες του συστήματος: την FIFO ουρά ασθενών στα επείγοντα και την παρακολούθηση υποστελεχωμένων βαρδιών.

View	Περιγραφή
ActiveStaff	Φιλτράρει τον πίνακα Staff με WHERE IsActive = 1 και υπολογίζει ταυτόχρονα την ηλικία ως TIMESTAMPDIFF(YEAR, BirthDate, CURDATE()). Η ηλικία δεν αποθηκεύεται στον πίνακα για να μην γίνεται stale — παράγεται on-the-fly από το view. Χρησιμοποιείται ευρέως ως βάση για queries που αφορούν ενεργό προσωπικό.
ActivePatient	Αντίστοιχο φίλτρο IsActive = 1 για τους ασθενείς. Επιστρέφει τα δημογραφικά στοιχεία και τον ασφαλιστικό φορέα, αποκρύπτοντας ανενεργές εγγραφές από τα queries κλινικής διαχείρισης.
ActiveLabTest	Φιλτράρει τον κατάλογο εξετάσεων (LabTest) με IsActive = 1. Διατηρεί παλιές εγγραφές στον πίνακα για ιστορικούς λόγους, αλλά εκθέτει μόνο τις τρέχουσες στα queries εξετάσεων.
DocInfo	Ενώνει Doctor με ActiveStaff και επιστρέφει συγκεντρωτικά όλα τα στοιχεία ενεργού ιατρού: AMKA, ονοματεπώνυμο, ηλικία, email, ημερομηνία πρόσληψης, άδεια, ειδικότητα, βαθμίδα και AMKA επόπτη. Αποφεύγει επαναλαμβανόμενο JOIN σε κάθε query που χρειάζεται πλήρη στοιχεία ιατρού, και χρησιμοποιείται ευρέως στο κεφάλαιο B.
DepartmentInfo	Ενώνει Department, Room (LEFT JOIN) και Staff (LEFT JOIN στο DirectorAMKA) και επιστρέφει ανά τμήμα: DepartmentID, όνομα, όροφο, κτίριο, επώνυμο Διευθυντή και αριθμό κλινών (COUNT(Room.ID)). Χρησιμοποιείται σε queries γενικής πληροφόρησης τμημάτων.
ShiftsAlerts	Εντοπίζει βάρδιες με ανεπαρκή στελέχωση. Χρησιμοποιεί CTE για να μετρήσει ανά βάρδια τον αριθμό ιατρών, νοσηλευτών και διοικητικών, και επιστρέφει μόνο τις βάρδιες που παραβιάζουν κάποιο ελάχιστο όριο (< 3 ιατροί, < 6 νοσηλευτές, < 2 διοικητικοί). Το πεδίο WarningReason περιγράφει ποιο ή ποια όρια δεν ικανοποιούνται, παρέχοντας άμεση εικόνα χωρίς ανάγκη aggregation query.
PatientQueue	Υλοποιεί την αναγνώσιμη FIFO ουρά επειγόντων: φιλτράρει τα TriageEvent με Outcome = 'Pending', κάνει JOIN με τον Patient για τα στοιχεία ασθενούς, και ταξινομεί κατά EmergencyLevel ASC, TriageDateTime ASC. Έτσι ο πρώτος στη λίστα είναι πάντα το πιο επείγον περιστατικό και, για το ίδιο επίπεδο, ο ασθενής που ήρθε νωρίτερα. Η stored procedure FetchNext χτίζει πάνω σε αυτό το view για να επιστρέφει τον επόμενο ασθενή στην ουρά έτσι ώστε να εξυπηρετηθεί.

A.4 Load και Synthetic Data

Η φόρτωση οργανώνεται σε δύο φάσεις. Πρώτα εκτελούνται τρία Python scripts που εξάγουν τους επίσημους καταλόγους αναφοράς από τις πηγές του Υπουργείου Υγείας και του EMA, παράγοντας τα αντίστοιχα CSVs στον φάκελο data/. Στη συνέχεια, το generate_synthetic_data.py παράγει τις λειτουργικές εγγραφές (προσωπικό, ασθενείς, νοσηλείες, εφημερίες κ.λπ.), σεβόμενο τους επιχειρησιακούς κανόνες κατά την παραγωγή. Τελικά, το sql/load.sql φορτώνει όλα τα CSVs με LOAD DATA LOCAL INFILE σε σειρά που σέβεται τις εξαρτήσεις FK.

Script	Περιγραφή
preprocess_procedure_types.py	Εξάγει ενότητες Α (Θεραπευτικές) και Β (Χειρουργικές) από το επίσημο αρχείο Ιατρικών Πράξεων. Κόστος και διάρκεια παράγονται αναπαραγωγίμα βάσει κωδικού και λέξεων-κλειδιών.
preprocess_lab_tests.py	Εξάγει ενότητες Γ (Απεικονιστικές), Δ (Βιοπαθολογικές) και Ε (Παθολογοανατομικές) από το ίδιο αρχείο. Εγγραφές με «ΑΝΕΝΕΡΓΟΣ» λαμβάνουν IsActive=0.
preprocess_article57_drugs.py	Διαβάζει το EMA Article 57.xlsx. Διαχωρίζει τις δραστικές ουσίες (separator) σε ξεχωριστές εγγραφές στον Substances και συνδέει κάθε φάρμακο με τις ουσίες του μέσω HasSubstances.
generate_synthetic_data.py	Παράγει όλες τις λειτουργικές εγγραφές διαβάζοντας τους επίσημους καταλόγους. Γράφει dataset_summary.json που επαληθεύει τα minimums της εκφώνησης.

Nullable πεδία (SupervisorAMKA, ExitDateTime, EndDate κ.λπ.) φορτώνονται μέσω user variables με NULLIF(@var, ""). Εξαίρεση αποτελεί ο DoctorEvaluation: φορτώνεται πρώτα σε προσωρινό πίνακα και εισάγεται στον τελικό μόνο αν υπάρχει αντίστοιχη εγγραφή στον PrescriptionEvent, ώστε να αξιολογείται μόνο ιατρός που πράγματι συνταγογράφησε.

Σύνοψη Εγγραφών:

Πίνακας	Πλήθος		Πίνακας	Πλήθος
Diagnosis	11.008		Staff	1.020 (300 γιατροί, 540 νοσηλευτές, 180 admin staff)
Cost (KEN)	701		Department	15
ProcedureType	5.761		Room	750
LabTest	2.836		Patient	900
DrugType	161.785		Hospitalization	3600
Substances	10.730		TriageEvent	4200
HasSubstances	258.680		HospLabTest	2600
DrugSupportPhone	156.108		ProcedureEvent	1200
Shift	315		PrescriptionEvent	3000
hasDoctor/Nurse/Admin	3.465		ProcedureRoom	20
HospEvaluation	1.841		allergic_to	439

B. Queries

B.1

Βρείτε τα συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος, με ανάλυση ανά KEN κωδικό (βασικό κόστος έναντι πρόσθετης χρέωσης λόγω υπέρβασης ΜΔΝ) και κατανομή νοσηλειών ανά ασφαλιστικό φορέα.

SQL

```
SELECT
  d.Name,
  YEAR(h.ExitDateTime) AS `Year`,
  c.KENCode,
  SUM(c.BaseCost) AS BaseCost,
  SUM(
    CASE
      WHEN h.ActualDays > c.PredictedAvgTime
      THEN (h.ActualDays - c.PredictedAvgTime) * c.ChargePerDay
      ELSE 0
    END
  ) AS ExtraCost,
  SUM(
    CASE
      WHEN h.ActualDays > c.PredictedAvgTime
```

```

        THEN (h.ActualDays - c.PredictedAvgTime) * c.ChargePerDay
        ELSE 0
    END
) + SUM(c.BaseCost) AS TotalRevenue,
p.InsuranceProviderName,
COUNT(h.HospitalizationID) AS HospitalizationCount
FROM Hospitalization h
JOIN Department d ON h.DepartmentID = d.DepartmentID
JOIN Cost c ON h.KENCode = c.KENCode
JOIN Patient p ON h.PatientAMKA = p.AMKA
WHERE h.ExitDateTime IS NOT NULL
GROUP BY
    d.DepartmentID,
    d.Name,
    'Year',
    c.KENCode,
    p.InsuranceProviderName;

```

Χρησιμοποιήσαμε την εντολή CASE για να ξεχωρίσουμε το βασικό κόστος της κάθε νοσηλείας από τις επιπλέον χρεώσεις που προστίθενται όταν ο ασθενής μένει παραπάνω μέρες από τις προβλεπόμενες (ΜΔΝ). Για να συγκεντρώσουμε αυτά τα στοιχεία, ενώνουμε (JOIN) τους πίνακες Hospitalization, Department, Cost και Patient, κρατώντας αυστηρά μόνο τις νοσηλείες που έχουν πάρει εξιτήριο (ExitDateTime IS NOT NULL). Τέλος, με την εντολή GROUP BY, ομαδοποιούμε και παρουσιάζουμε τα έσοδα ανά τμήμα, έτος, κωδικό KEN και ασφαλιστικό φορέα.

B.2

Για συγκεκριμένη ειδικότητα ιατρού, βρείτε όλους τους ιατρούς που ανήκουν σε αυτήν, με ένδειξη αν είχαν εφημερία το τρέχον έτος και πόσες επεμβάσεις εκτέλεσαν ως κύριοι χειρουργοί.

SQL

```

WITH ShiftCnt AS (
    SELECT DoctorAMKA, COUNT(*) AS ShiftTotal
    FROM hasDoctor
    WHERE YEAR(ShiftDate) = YEAR(CURDATE())
    GROUP BY DoctorAMKA
),
SurgCnt AS (
    SELECT pev.MainDocAMKA, COUNT(pev.ProcEventID) AS SurgTotal
    FROM ProcedureEvent pev
    JOIN ProcedureType pt ON pev.ProcedureCode = pt.ProcCode
    WHERE pt.ProcType = 'Surgical'
    AND YEAR(pev.DateTime) = YEAR(CURDATE())
    GROUP BY pev.MainDocAMKA
)
SELECT d.AMKA, d.FirstName, d.LastName,
CASE WHEN COALESCE(she.ShiftTotal, 0) > 0 THEN 'yes' ELSE 'no' END AS HadShift,

```

```

COALESCE(sc.SurgTotal, 0) AS TotalProcedures
FROM DocInfo d
LEFT JOIN ShiftCnt shc ON d.AMKA = shc.DoctorAMKA
LEFT JOIN SurgCnt sc ON d.AMKA = sc.MainDocAMKA
WHERE d.Specialty = 'Cardiology'; -- Παράδειγμα ειδικότητας

```

Για την εξαγωγή των δεδομένων, χρησιμοποιήσαμε Common Table Expressions (CTE) ώστε να προ-υπολογίσουμε το πλήθος των βαρδιών και των χειρουργικών επεμβάσεων ανά ιατρό για το τρέχον έτος. Στη συνέχεια, αξιοποιώντας την όψη *DocInfo*, εκτελούμε αριστερές εξωτερικές συνενώσεις (LEFT JOIN) για να συμπεριλάβουμε το σύνολο των ιατρών της ζητούμενης ειδικότητας, ακόμη και αν δεν είχαν καμία δραστηριότητα. Με τη χρήση της συνάρτησης COALESCE, διασφαλίζουμε ότι οι μηδενικές συμμετοχές θα εμφανίζονται ορθά ως αριθμητικές τιμές και όχι ως κενά πεδία.

B.3.

Βρείτε ποιοι ασθενείς έχουν νοσηλευτεί περισσότερες από 3 φορές στο ίδιο τμήμα, με το συνολικό κόστος νοσηλείας τους.

```

SQL

SELECT
  p.FirstName,
  p.LastName,
  d.`Name`,
  COUNT(h.HospitalizationID) AS num_of_visits,
  SUM(
    c.BaseCost +
    (GREATEST(DATEDIFF(IFNULL(h.ExitDateTime, CURDATE()), h.AdmissionDateTime) -
    c.PredictedAvgTime, 0)
    * c.ChargePerDay)
  ) AS total_cost

FROM Hospitalization h
JOIN Patient p ON h.PatientAMKA = p.AMKA
JOIN Department d ON h.DepartmentID = d.DepartmentID
JOIN Cost c ON h.KENcode = c.KENcode

GROUP BY p.AMKA, p.FirstName, p.LastName, d.`Name`
HAVING COUNT(h.HospitalizationID) > 3;

```

Για την εξαγωγή της συγκεκριμένης πληροφορίας, πραγματοποιήσαμε συνένωση (JOIN) των πινάκων Hospitalization, Patient, Department και Cost, ώστε να συσχετίσουμε τα δημογραφικά στοιχεία των ασθενών με το ιστορικό των επισκέψεών τους και τα αντίστοιχα κόστη. Για τον ακριβή υπολογισμό των εσόδων, χρησιμοποιήσαμε τη συνάρτηση GREATEST προκειμένου να επιβάλουμε την πρόσθετη ημερήσια χρέωση μόνο στις περιπτώσεις που η πραγματική διάρκεια νοσηλείας υπερβαίνει την προβλεπόμενη ΜΑΝ. Τέλος, εφαρμόσαμε ομαδοποίηση (GROUP BY) ανά ασθενή και τμήμα,

θέτοντας τον περιορισμό `HAVING COUNT > 3` για τον εντοπισμό εκείνων των περιπτώσεων με αυξημένη (>3) συχνότητα νοσηλείας στην ίδια κλινική.

B.4

Για συγκεκριμένο ιατρό, βρείτε τον μέσο όρο αξιολογήσεων των ασθενών του (κριτήριο: Ποιότητα ιατρικής φροντίδας) και τη Συνολική εντύπωση νοσηλείας.

SQL

```
SELECT
  de.DoctorAMKA,
  s.FirstName,
  s.LastName,
  ROUND(AVG(de.QoDoctorS), 2) AS AvgDoctorQuality,
  ROUND(AVG(he.GeneralExperience), 2) AS AvgHospitalExperience
FROM DoctorEvaluation de
JOIN HospEvaluation he ON de.HospitalizationID = he.HospitalizationID
JOIN ActiveStaff s ON de.DoctorAMKA = s.AMKA
WHERE de.DoctorAMKA = '1000000000' -- Παράδειγμα ιατρού
GROUP BY de.DoctorAMKA, s.FirstName, s.LastName;
```

Πραγματοποιήσαμε συνένωση (JOIN) των πινάκων DoctorEvaluation, HospEvaluation και της όψης ActiveStaff, ώστε να αντιστοιχίσουμε τις αξιολογήσεις των ασθενών με τα προσωπικά στοιχεία του ιατρού. Χρησιμοποιήσαμε τη συνάρτηση AVG για την εξαγωγή του μέσου όρου στην ποιότητα φροντίδας και στη συνολική εμπειρία νοσηλείας, εφαρμόζοντας παράλληλα τη συνάρτηση ROUND για τον περιορισμό των αποτελεσμάτων σε δύο δεκαδικά ψηφία. Τέλος, μέσω της ομαδοποίησης (GROUP BY), απομονώσαμε τα στατιστικά στοιχεία για τον συγκεκριμένο ιατρό, διασφαλίζοντας την ακρίβεια της αναφοράς ανά επαγγελματία υγείας.

A. EXPLAIN / EXPLAIN ANALYZE

Αποτελέσματα EXPLAIN

Παρατηρούμε πως μέσω της αποκλειστικής χρήσης ευρετηρίων (Index-based lookups), εξαλείφεται πλήρως η ανάγκη για κοστοβόρες πλήρεις σαρώσεις πινάκων (Full Table Scans):

Id	Select_type	Table	Partitions	Type	Possible_keys	Key	Key_len	Ref	Rows	Extra
1	SIMPLE	staff	NULL	const	PRIMARY	PRIMARY	44	const	1	NULL
1	SIMPLE	he	NULL	eq_ref	PRIMARY	PRIMARY	4	hospitaldb.de.HospitalizationID	1	NULL
1	SIMPLE	de	NULL	ref	PRIMARY,DoctorAMKA	DoctorAMKA	44	const	12	Using index cc

Αρχικά, ο απαιτούμενος ιατρός απομονώνεται από την όψη `ActiveStaff` σε θεωρητικά σταθερό χρόνο $O(1)$ (**type: const**) μέσω του Πρωτεύοντος Κλειδιού (AMKA), ελαχιστοποιώντας άμεσα το μέγεθος του ενδιάμεσου αποτελέσματος. Στο δεύτερο στάδιο, η μηχανή αποθήκευσης (InnoDB) αξιοποιεί το δευτερεύον ευρετήριο `DoctorAMKA` του πίνακα `DoctorEvaluation` για να εντοπίσει στοχευμένα (**type: ref**) τις 12 σχετικές εγγραφές. Εδώ, η εφαρμογή της προηγμένης τεχνικής `Index Condition Pushdown` (**Using index condition**) επιτρέπει το φιλτράρισμα χαμηλά στο επίπεδο του `Storage Engine`, μειώνοντας δραματικά τις περιττές I/O προσπελάσεις στον δίσκο. Τέλος, για καθεμία από τις 12 αυτές εγγραφές, η συνένωση με τον πίνακα `HospEvaluation` εκτελείται μέσω του Πρωτεύοντος Κλειδιού (`HospitalizationID`), διαδικασία που αξιοποιεί αποδοτική μέθοδο σύζευξης της SQL (**type: eq_ref**). Η παντελής απουσία προσωρινών δομών μνήμης (`Using temporary`) ή δαπανηρών εξωτερικών ταξινομήσεων (`Using filesort`) επιβεβαιώνει την αρτιότητα του φυσικού σχεδιασμού και τη μέγιστη απόδοση του ερωτήματος.

Αποτελέσματα EXPLAIN ANALYZE

Σε αντίθεση με την παραδοσιακή εντολή `EXPLAIN` η οποία παρέχει στατιστικές εκτιμήσεις (`Cost Estimates`), η εντολή `EXPLAIN ANALYZE` εκτελεί πραγματικά το ερώτημα και καταγράφει τον ακριβή χρόνο (σε ms), τον όγκο δεδομένων και τις επαναλήψεις του αλγορίθμου:

EXPLAIN:

```
-> Group aggregate: avg(he.GeneralExperience), avg(de.QoDoctorS) (cost=9.6 rows=9.36) (actual time=0.233..0.233 rows=1 loops=1)
-> Nested loop inner join (cost=8.4 rows=12) (actual time=0.173..0.224 rows=12 loops=1)
-> Index lookup on de using DoctorAMKA (DoctorAMKA='10000000000'), with index condition: (de.DoctorAMKA = '10000000000') (cost=4.2 rows=12) (actual time=0.12..0.123 rows=12 loops=1)
-> Single-row index lookup on he using PRIMARY (HospitalizationID=de.HospitalizationID) (cost=0.258 rows=1) (actual time=0.00805..0.00808 rows=1 loops=12)
```

Αρχικά, στη βάση του δέντρου, το σύστημα εκτελεί `Index Lookup` αξιοποιώντας την τεχνική **Index Condition Pushdown** στον πίνακα των αξιολογήσεων του ιατρού (`de`), εντοπίζοντας τις 12 σχετικές εγγραφές σε μόλις 0.12 ms. Στη συνέχεια, η μετρική `loops=12` αποδεικνύει την επαναληπτική λειτουργία του εσωτερικού βρόχου (`inner loop`), όπου για καθεμία από τις 12 εγγραφές εκτελείται μια ακαριαία αναζήτηση Πρωτεύοντος Κλειδιού (**Single-row index lookup**) στον πίνακα των νοσηλειών (`he`), δαπανώντας μόλις 0.008 ms ανά επανάληψη. Τα συλλεχθέντα δεδομένα προωθούνται τελικά στον τελεστή συνάθροισης (`Group aggregate`) για τον υπολογισμό των μέσων όρων, ολοκληρώνοντας την παραγωγή της τελικής εγγραφής σε συνολικό χρόνο 0.233 ms.

Σημείωση: Ο πίνακας `ActiveStaff` έχει ήδη αξιολογηθεί σε μηδενικό χρόνο ως σταθερά (constant folding) στο `Server Layer` λόγω της αναζήτησης με `PRIMARY KEY`, γι' αυτό και το δέντρο εστιάζει αποκλειστικά στον "βαρύ" πυρήνα του `JOIN` μεταξύ των αξιολογήσεων.

B. FORCE INDEX

Τροποποιήσαμε το Q4, ώστε να υλοποιήσουμε αυτή την ζητούμενη εκδοχή, τοποθετώντας την εντολή **FORCE INDEX (PRIMARY)** για τον 'de' πίνακα, κατά το JOIN του με το **ACTIVESTAFF**:

```
SQL
FROM DoctorEvaluation de FORCE INDEX (PRIMARY)
```

Παρακάτω τα αποτελέσματα του EXPLAIN ANALYZE:

```
EXPLAIN:
-> Group aggregate: avg(he.GeneralExperience), avg(de.QoDoctorS) (cost=1014 rows=868) (actual time=3.62..3.62 rows=1 loops=1)
-> Nested loop inner join (cost=830 rows=1841) (actual time=0.83..3.6 rows=12 loops=1)
-> Table scan on he (cost=185 rows=1841) (actual time=0.196..0.56 rows=1841 loops=1)
-> Filter: (de.DoctorAMKA = '10000000000') (cost=0.25 rows=1) (actual time=0.00157..0.00157 rows=0.00652 loops=1841)
```

Παρατηρούμε πως με τη χρήση του **FORCE INDEX (PRIMARY)**, η μηχανή δεν μπόρεσε να φιλτράρει απευθείας τον ιατρό, καθώς το πεδίο **DoctorAMKA** αποτελεί το δεύτερο συνθετικό του πρωτεύοντος κλειδιού, παραβιάζοντας τον κανόνα του αριστερότερου προθέματος (Leftmost Prefix Rule). Αυτό εξανάγκασε τον Optimizer να αντιστρέψει τη σειρά συνένωσης, εκτελώντας μια εξαιρετικά δαπανηρή πλήρη σάρωση (**Table scan**) 1841 εγγραφών στον πίνακα **HospEvaluation** και ισάριθμες αναζητήσεις (**loops=1841**) στον πίνακα **DoctorEvaluation**.

C. ESTIMATED COST VS ACTUAL EXECUTION TIME

- **Χωρίς FORCE INDEX (Βέλτιστο πλάνο):** Εκτιμώμενο Κόστος: 9.60 | Πραγματικός Χρόνος: 0.233 ms
- **Με FORCE INDEX (Εναλλακτικό πλάνο):** Εκτιμώμενο Κόστος: 1014.00 | Πραγματικός Χρόνος: 3.62 ms

D. ANALYSIS OF ESTIMATED COST & ACTUAL TIME

Στη βέλτιστη εκτέλεση, ο Query Optimizer αξιοποίησε το **δευτερεύον ευρετήριο DoctorAMKA** για να ανακτήσει στοχευμένα μόνο τις 12 αξιολογήσεις του ιατρού (Index Lookup), διατηρώντας το κόστος ελάχιστο. Αντίθετα, με τη χρήση του **FORCE INDEX (PRIMARY)**, η μηχανή δεν μπόρεσε να φιλτράρει απευθείας τον ιατρό, καθώς το πεδίο **DoctorAMKA** αποτελεί το δεύτερο συνθετικό του πρωτεύοντος κλειδιού, παραβιάζοντας τον κανόνα του αριστερότερου προθέματος (Leftmost Prefix Rule).

Αυτό εξανάγκασε τον Optimizer να αντιστρέψει τη σειρά συνένωσης, εκτελώντας μια εξαιρετικά δαπανηρή πλήρη σάρωση (**Table scan**) 1841 εγγραφών στον πίνακα **HospEvaluation** και ισάριθμες αναζητήσεις (**loops=1841**) στον πίνακα **DoctorEvaluation**. Η ραγδαία αύξηση του εκτιμώμενου

κόστους (από 9.6 σε 1014) και ο 15-πλασιασμός του πραγματικού χρόνου εκτέλεσης (από 0.233 ms σε 3.62 ms) αποδεικνύουν εμπράκτως τη σχεδιαστική αναγκαιότητα δημιουργίας δευτερευόντων ευρετηρίων σε πεδία ξένων κλειδιών για την αποφυγή περιττών σαρώσεων, κάτι για το οποίο φροντίζει η MySQL, φτιάχνοντας δευτερεύοντα ευρετήρια τόσο στα foreign keys όσο και στα δηλωμένα unique set of attributes.

B.5

Βρείτε τους νέους ιατρούς (ηλικία < 35 ετών) που έχουν εκτελέσει τις περισσότερες χειρουργικές επεμβάσεις ως κύριοι χειρουργοί.

SQL

```
SELECT
  d.AMKA,
  d.FirstName,
  d.LastName,
  d.Age,
  COUNT(pev.ProcEventID) AS TotalSurgicalProc
FROM DocInfo d
JOIN ProcedureEvent pev ON pev.MainDocAMKA = d.AMKA
JOIN ProcedureType pt ON pt.ProcCode = pev.ProcedureCode
WHERE pt.ProcType = 'Surgical'
AND d.Age < 35
GROUP BY d.AMKA, d.FirstName, d.LastName, d.Age
ORDER BY TotalSurgicalProc DESC;
```

Πραγματοποιήσαμε συνένωση (JOIN) της όψης DocInfo με τους πίνακες ProcedureEvent και ProcedureType, ώστε να συσχετίσουμε τα δημογραφικά στοιχεία του ιατρικού προσωπικού με το αρχείο των επεμβάσεων. Εφαρμόσαμε κριτήρια φιλτραρίσματος (WHERE) για τον περιορισμό του δείγματος σε ιατρούς ηλικίας κάτω των 35 ετών και την επιλογή αποκλειστικά χειρουργικών πράξεων. Με τη χρήση της συνάρτησης COUNT και της ομαδοποίησης (GROUP BY) ανά AMKA, υπολογίσαμε το συνολικό κλινικό έργο ανά ιατρό, ταξινομώντας (ORDER BY) τα αποτελέσματα σε φθίνουσα σειρά για την ανάδειξη των πιο δραστήριων νέων χειρουργών του νοσοκομείου.

B.6

Για συγκεκριμένο ασθενή, βρείτε το ιστορικό νοσηλείών του, τις αντίστοιχες διαγνώσεις (ICD-10), το συνολικό κόστος ανά νοσηλεία και τον μέσο όρο αξιολόγησής του.

SQL

```
SELECT
  h.HospitalizationID,
  h.AdmissionDateTime,
```

```

h.ExitDateTime,
(SELECT GROUP_CONCAT(DISTINCT CONCAT(d.ICDcode, ': ', d.Description) SEPARATOR ' | '))
FROM AdmissionDiagnosis ad
JOIN Diagnosis d ON ad.ICDcode = d.ICDcode
WHERE ad.HospitalizationID = h.HospitalizationID) AS ICD_diagnoses,
(c.BaseCost +
GREATEST(IFNULL(h.ActualDays, DATEDIFF(CURDATE(), h.AdmissionDateTime)) - c.PredictedAvgTime, 0)
* c.ChargePerDay) AS TotalCost,
(SELECT (dq.AvgDoctorService + he.QoNurseS + he.Cleanliness + he.Food + he.GeneralExperience)
/ 5.0
FROM HospEvaluation he
JOIN (
SELECT HospitalizationID, AVG(QoDoctorS) AS AvgDoctorService
FROM DoctorEvaluation
GROUP BY HospitalizationID
) dq USING (HospitalizationID)
WHERE he.HospitalizationID = h.HospitalizationID) AS AvgRating
FROM Hospitalization h
JOIN Cost c ON h.KENcode = c.KENcode
WHERE h.PatientAMKA = '30000000000' -- Παράδειγμα ασθενούς
ORDER BY h.AdmissionDateTime;

```

Για την ανάκτηση της πλήρους κλινικής εικόνας του ασθενούς, συνδυάσαμε δεδομένα από τους πίνακες Hospitalization, Cost και Diagnosis. Χρησιμοποιήσαμε subqueries και τη συνάρτηση GROUP_CONCAT για τη συγκέντρωση των διαγνώσεων ICD-10 σε μία ενιαία συμβολοσειρά ανά νοσηλεία. Ο υπολογισμός του κόστους βασίστηκε στο μοντέλο των KEN, ενσωματώνοντας το βασικό νοσήλιο και τις πρόσθετες χρεώσεις υπερημερίας μέσω των συναρτήσεων GREATEST και IFNULL. Παράλληλα, εξάγαμε το AvgRating συνδυάζοντας την ποιότητα της ιατρικής και νοσηλευτικής φροντίδας με την εμπειρία των υποδομών. Τέλος, τα αποτελέσματα ταξινομήθηκαν χρονολογικά (ORDER BY)

A. EXPLAIN / EXPLAIN ANALYSE

Αποτελέσματα EXPLAIN

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	h	None	ref	PatientAMKA,KENcode,idx_1	PatientAMKA	44	const	9	100	Using index condition
1	PRIMARY	c	None	eq_ref	PRIMARY	PRIMARY	22	hospitaldb.h.KENcode	1	100	None
3	DEPENDENT SUBQUERY	he	None	eq_ref	PRIMARY	PRIMARY	4	hospitaldb.h.HospitalizationID	1	100	None
3	DEPENDENT SUBQUERY	<derived4>	None	ref	<auto_key0>	<auto_key0>	4	hospitaldb.h.HospitalizationID	253	100	Using where
4	DERIVED	DoctorEvaluation	None	index	PRIMARY,DoctorAMKA	PRIMARY	48	None	2530	100	None
2	DEPENDENT SUBQUERY	ad	None	ref	PRIMARY,ICDCode	PRIMARY	4	hospitaldb.h.HospitalizationID	1	100	Using index
2	DEPENDENT SUBQUERY	d	None	eq_ref	PRIMARY	PRIMARY	42	hospitaldb.ad.ICDCode	1	100	None

Στο πρωτεύον εξωτερικό ερώτημα (id = 1), ο Query Optimizer ανακτά αρχικά τις νοσηλείες του ασθενούς (πίνακας h) με γρήγορη προσπέλαση ref μέσω του ευρετηρίου PatientAMKA. Εδώ αξιοποιείται η τεχνική Index Condition Pushdown (ICP) (Using index condition), η οποία επιτρέπει στη μηχανή InnoDB να φιλτράρει τα δεδομένα απευθείας στα φύλλα του ευρετηρίου, ελαχιστοποιώντας τα δαπανηρά Random I/Os. Στη συνέχεια, η συνένωση (JOIN) με τον πίνακα κόστους (c) εκτελείται με τη βέλτιστη δυνατή μέθοδο eq_ref, διαβάζοντας ταχύτατα ακριβώς μία εγγραφή κόστους για κάθε νοσηλεία μέσω του Πρωτεύοντος Κλειδιού.

Στο συσχετιζόμενο υποερώτημα (id = 2) για την ανάκτηση των διαγνώσεων (ICD-10), ο Query Optimizer προσπελαίνει τον πίνακα AdmissionDiagnosis (ad) με ref lookup. Εδώ κυριαρχεί η αξιοποίηση ενός Covering Index (Index-Only Scan), καθώς η ένδειξη Using index υποδηλώνει ότι η βάση ανακτά τα δεδομένα αποκλειστικά από τις σελίδες του ευρετηρίου, παρακάμπτοντας εντελώς την ανάγνωση των φυσικών blocks του πίνακα. Τέλος, οι περιγραφές των διαγνώσεων (πίνακας d) συνενώνονται με τη βέλτιστη δυνατή μέθοδο eq_ref μέσω του Πρωτεύοντος Κλειδιού τους, εξασφαλίζοντας ακαριαία ανάκτηση σε σταθερό χρόνο O(1).

Στο τελικό στάδιο υπολογισμού της αξιολόγησης (id = 3 & 4), ο Query Optimizer συνδυάζει τις βαθμολογίες ιατρού και νοσοκομείου. Αρχικά, δημιουργεί έναν προσωρινό πίνακα (derived) για να υπολογίσει τον μέσο όρο του ιατρού, σαρώνοντας αποδοτικά μόνο το ευρετήριο (index scan) (Covering Index). Στη συνέχεια, ανακτά ακαριαία την αξιολόγηση του νοσοκομείου (HospEvaluation) κάνοντας χρήση του Πρωτεύοντος Κλειδιού της (eq_ref). Τέλος, ενώνει ταχύτατα αυτά τα δύο αποτελέσματα αξιοποιώντας ένα εσωτερικό κλειδί (<auto_key0>) που παράγει αυτόματα το ίδιο το σύστημα.

Αποτελέσματα EXPLAIN ANALYZE

```
-> Nested loop inner join (cost=6.3 rows=9) (actual time=0.176..0.288 rows=9 loops=1)
  -> Filter: (h.KENcode is not null) (cost=3.15 rows=9) (actual time=0.14..0.154 rows=9 loops=1)
  -> Index lookup on h using PatientAMKA (PatientAMKA='30000000871'), with index condition: (h.PatientAMKA = '30000000871') (cost=3.15 rows=9) (actual time=0.139..0.151 rows=9 loops=1)
  -> Single-row index lookup on c using PRIMARY (KENCode=h.KENcode) (cost=0.261 rows=1) (actual time=0.0144..0.0144 rows=1 loops=9)
-> Select #2 (subquery in projection; dependent)
  -> Aggregate: group_concat(distinct concat(d.ICDCode;', 'd.' Description `) separator ' | ' ) (cost=0.8 rows=1) (actual time=0.0315..0.0316 rows=1 loops=9)
  -> Nested loop inner join (cost=0.7 rows=1) (actual time=0.0264..0.0273 rows=1 loops=9)
  -> Covering index lookup on ad using PRIMARY (HospitalizationID=h.HospitalizationID) (cost=0.35 rows=1) (actual time=0.00945..0.0103 rows=1 loops=9)
  -> Single-row index lookup on d using PRIMARY (ICDCode=ad.ICDCode) (cost=0.35 rows=1) (actual time=0.0167..0.0168 rows=1 loops=9)
-> Select #3 (subquery in projection; dependent)
  -> Nested loop inner join (cost=88.9 rows=253) (actual time=0.402..0.402 rows=0.222 loops=9)
  -> Single-row index lookup on he using PRIMARY (HospitalizationID=h.HospitalizationID) (cost=0.35 rows=1) (actual time=0.0084..0.0084 rows=0.333 loops=9)
  -> Filter: (dq.HospitalizationID = he.HospitalizationID) (cost=675..88.5 rows=253) (actual time=1.18..1.18 rows=0.667 loops=3)
  -> Index lookup on dq using <auto_key0> (HospitalizationID=h.HospitalizationID) (cost=678..766 rows=253) (actual time=1.18..1.18 rows=0.667 loops=3)
  -> Materialize (cost=677..677 rows=1693) (actual time=3.52..3.52 rows=1693 loops=1)
  -> Group aggregate: avg(doctorevaluation.QoDoctorS) (cost=508 rows=1693) (actual time=0.0804..1.72 rows=1693 loops=1)
  -> Index scan on DoctorEvaluation using PRIMARY (cost=255 rows=2530) (actual time=0.0772..1.18 rows=2530 loops=1)
```

Το EXPLAIN ANALYZE επιβεβαιώνει ότι το πλάνο του απλού EXPLAIN εκτελείται αποδοτικά στην πράξη. Η ανάκτηση των νοσηλειών του συγκεκριμένου ασθενούς γίνεται με index lookup στον πίνακα Hospitalization, ενώ η σύνδεση με τον πίνακα Cost γίνεται με single-row lookup μέσω του πρωτεύοντος κλειδιού. Οι πραγματικές τιμές rows/time είναι μικρές, όπως αναμένεται, επειδή το φίλτρο PatientAMKA περιορίζει νωρίς το αποτέλεσμα.

Στο υποερώτημα των ICD-10 διαγνώσεων, το AdmissionDiagnosis διαβάζεται μέσω covering index για κάθε νοσηλεία και οι περιγραφές ανακτώνται από το Diagnosis μέσω primary-key lookup. Για τις αξιολογήσεις, ο optimizer υλοποιεί το derived table των μέσων βαθμολογιών γιατρών και το συνδέει με το HospEvaluation μέσω εσωτερικού auto key. Συνολικά, η εκτέλεση επιβεβαιώνει ότι τα διαθέσιμα indexes περιορίζουν τις άσκοπες σαρώσεις και ότι το κόστος του Q6 εξαρτάται κυρίως από το πλήθος νοσηλειών του συγκεκριμένου ασθενή.

B. Αποτελέσματα FORCE INDEX

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	h	None	ALL	None	None	None	None	3600	0.12	Using where; Using
1	PRIMARY	c	None	eq_ref	PRIMARY	PRIMARY	22	hospitaldb.h.KENcode	1	100	None
3	DEPENDENT SUBQUERY	he	None	eq_ref	PRIMARY	PRIMARY	4	hospitaldb.h.HospitalizationID	1	100	None
3	DEPENDENT SUBQUERY	<derived4>	None	ref	<auto_key0>	<auto_key0>	4	hospitaldb.h.HospitalizationID	253	100	Using where
4	DERIVED	DoctorEvaluation	None	index	PRIMARY,DoctorAMKA	PRIMARY	48	None	2530	100	None
2	DEPENDENT SUBQUERY	ad	None	ref	PRIMARY,ICDCode	PRIMARY	4	hospitaldb.h.HospitalizationID	1	100	Using index
2	DEPENDENT SUBQUERY	d	None	eq_ref	PRIMARY	PRIMARY	42	hospitaldb.ad.ICDCode	1	100	None

Προκειμένου να αξιολογηθεί η συμπεριφορά του Query Optimizer, τροποποιήσαμε το ερώτημα προσθέτοντας τη ρήτρα FORCE INDEX (PRIMARY) στον πίνακα Hospitalization, εξαναγκάζοντας το σύστημα να αγνοήσει το δευτερεύον ευρετήριο PatientAMKA που είχε επιλέξει αρχικά.

Το σύστημα αδυνατώντας να φιλτράρει το AMKA μέσω του Πρωτεύοντος Κλειδιού, κατέφυγε σε πλήρη σάρωση (Full Table Scan, type: ALL / index). Οι εκτιμώμενες εγγραφές (rows) εκτοξεύτηκαν στις 3.600 (το σύνολο του πίνακα), το κόστος αυξήθηκε δραματικά και εμφανίστηκε η ένδειξη Using where, καθώς το φιλτράρισμα έγινε εκ των υστέρων στη μνήμη.

C. ESTIMATED COST VS ACTUAL EXECUTION TIME

- **Χωρίς FORCE INDEX (Βέλτιστο πλάνο):** Εκτιμώμενο Κόστος: 6.3 | Πραγματικός Χρόνος: 0.264 ms
- **Με FORCE INDEX (Εναλλακτικό πλάνο):** Εκτιμώμενο Κόστος: 1621 | Πραγματικός Χρόνος: 4.37 ms

D. ANALYSIS OF ESTIMATED COST & ACTUAL TIME

Στη βέλτιστη εκτέλεση του Q6, ο Query Optimizer αξιοποίησε το ευρετήριο που ξεκινά από το PatientAMKA στον πίνακα Hospitalization, ώστε να ανακτήσει στοχευμένα μόνο τις νοσηλείες του συγκεκριμένου ασθενή και να τις επιστρέψει ήδη ταξινομημένες ως προς AdmissionDateTime. Η επιλογή αυτή είναι κατάλληλη, επειδή το βασικό φίλτρο του ερωτήματος είναι το h.PatientAMKA και το αποτέλεσμα ζητείται ως ιστορικό νοσηλειών.

Αντίθετα, με τη χρήση FORCE INDEX (PRIMARY), ο Optimizer εξαναγκάστηκε να χρησιμοποιήσει το πρωτεύον κλειδί του Hospitalization, δηλαδή το HospitalizationID. Το πεδίο αυτό δεν βοηθά στο φιλτράρισμα με βάση το PatientAMKA, οπότε η μηχανή κατέφυγε σε σάρωση ολόκληρου του πίνακα Hospitalization και εφάρμοσε το WHERE εκ των υστέρων. Αυτό φαίνεται και από την αύξηση των εκτιμώμενων rows στις 3.600 εγγραφές, δηλαδή στο πλήθος όλου του πίνακα.

Η διαφορά αποτυπώνεται καθαρά τόσο στο estimated cost όσο και στον πραγματικό χρόνο εκτέλεσης. Χωρίς FORCE INDEX, το κόστος ήταν 6.3 και ο χρόνος 0.264 ms, ενώ με FORCE INDEX (PRIMARY) το κόστος αυξήθηκε σε 1621 και ο χρόνος σε 4.37 ms. Άρα το εναλλακτικό πλάνο είναι περίπου 16.5 φορές πιο αργό στην πράξη και πολύ ακριβότερο σύμφωνα με τις εκτιμήσεις του optimizer.

Το αποτέλεσμα επιβεβαιώνει ότι για queries ιστορικού ασθενή είναι κρίσιμο να υπάρχει ευρετήριο με πρώτο γνώρισμα το PatientAMKA. Το index idx_pat_days_in_hospital(PatientAMKA, AdmissionDateTime, ExitDateTime) ταιριάζει στη μορφή του Q6, επειδή υποστηρίζει και το φιλτράρισμα ανά ασθενή και τη χρονολογική διάταξη των νοσηλειών. Η χρήση FORCE INDEX (PRIMARY) λειτουργεί εδώ ως αρνητικό παράδειγμα, δείχνοντας πόσο επιβαρύνεται το ερώτημα όταν ο optimizer στερείται το κατάλληλο access path.

B.7

Βρείτε για κάθε δραστική ουσία τον αριθμό ασθενών που έχουν δηλώσει αλλεργία και τον αριθμό φαρμάκων που την περιέχουν, ταξινομημένα κατά συνολικό αριθμό αλλεργικών ασθενών.

SQL

SELECT

s.`Name`,

COUNT(DISTINCT a.`PatientAMKA`) AS TotalPatients,

COUNT(DISTINCT h.`DrugID`) AS TotalDrugs

FROM `Substances` s

LEFT JOIN allergic_to a ON s.`ID` = a.`SubstanceID`

```
LEFT JOIN `HasSubstances` h ON h.`SubID` = s.`ID`
GROUP BY s.`ID`, s.`Name`
ORDER BY TotalPatients DESC;
```

Για την εξαγωγή των στατιστικών ανά δραστική ουσία, χρησιμοποιήσαμε *LEFT JOIN* μεταξύ των πινάκων Substances, allergic_to και HasSubstances. Η επιλογή αυτή διασφαλίζει ότι θα συμπεριληφθούν στο αποτέλεσμα όλες οι καταγεγραμμένες ουσίες, ακόμη και αν δεν συνδέονται με αλλεργίες. Με τη χρήση της συνάρτησης **COUNT(DISTINCT ...)**, επιτύχαμε τον μοναδικό υπολογισμό των ασθενών και των φαρμάκων ανά ουσία, αποφεύγοντας διπλές καταμετρήσεις λόγω των πολλαπλών συσχετίσεων. Τέλος, η ομαδοποίηση (GROUP BY) και η φθίνουσα ταξινόμηση (ORDER BY) επιτρέπουν την άμεση αναγνώριση των ουσιών με τη μεγαλύτερη επίπτωση στην ασφάλεια των ασθενών του νοσοκομείου.

B.8

Βρείτε το προσωπικό (ιατροί, νοσηλευτές, διοικητικό προσωπικό) που δεν έχει προγραμματισμένη εφημερία σε συγκεκριμένη ημερομηνία και τμήμα.

SQL

```
WITH nightshifts AS (
  SELECT hasdoctor.doctoramka AS amka
  FROM hasdoctor
  WHERE hasdoctor.shifttypename = 'Night'
  AND hasdoctor.shiftdate = '2024-01-01'
  AND hasdoctor.departmentid = (SELECT departmentid FROM department WHERE `name` = 'Cardiology')
  UNION ALL
  SELECT hasnurse.nurseamka AS amka
  FROM hasnurse
  WHERE hasnurse.shifttypename = 'Night'
  AND hasnurse.shiftdate = '2024-01-01'
  AND hasnurse.departmentid = (SELECT departmentid FROM department WHERE `name` = 'Cardiology')
  UNION ALL
  SELECT hasadmin.adminamka AS amka
  FROM hasadmin
  WHERE hasadmin.shifttypename = 'Night'
  AND hasadmin.shiftdate = '2024-01-01'
  AND hasadmin.departmentid = (SELECT departmentid FROM department WHERE `name` = 'Cardiology')
)
SELECT s.firstname, s.lastname, s.age, s.`type`
FROM activestaff s
WHERE NOT EXISTS (
  SELECT 1
  FROM nightshifts ns
```



```
WHERE ns.amka = s.amka
```

```
);
```

Για την υλοποίηση του ερωτήματος, χρησιμοποιήσαμε ένα **CTE** προκειμένου να συγκεντρώσουμε τα AMKA των ιατρών, νοσηλευτών και διοικητικών υπαλλήλων που εργάστηκαν στη νυχτερινή βάρδια (εφημερία) του επιλεγμένου τμήματος και ημερομηνίας. Μέσω της εντολής **UNION ALL**, ενοποιήσαμε τις συμμετοχές από τους τρεις πίνακες `hasDoctor`, `hasNurse`, `hasAdmin`. Τέλος, αντλώντας δεδομένα από την όψη *ActiveStaff*, εφαρμόσαμε τη συνθήκη **WHERE NOT EXISTS** για να απομονώσουμε το ενεργό προσωπικό που δεν περιλαμβανόταν στη συγκεκριμένη λίστα εφημερίας.

B.9

Βρείτε ποιοι ασθενείς νοσηλεύτηκαν τον ίδιο αριθμό ημερών σε διάστημα ενός έτους, με συνολική διάρκεια άνω των 15 ημερών.

SQL

```
WITH YearStats AS (  
  SELECT  
    PatientAMKA,  
    YEAR(AdmissionDateTime) AS HospitalYear,  
    SUM(ActualDays) AS TotalDays  
  FROM Hospitalization  
  GROUP BY PatientAMKA, YEAR(AdmissionDateTime)  
  HAVING TotalDays > 15  
)  
SELECT  
  ys.HospitalYear AS 'Year',  
  ys.TotalDays AS 'Total Hospitalization Days',  
  COUNT(ys.PatientAMKA) AS 'Number of Patients',  
  GROUP_CONCAT(CONCAT(p.FirstName, ' ', p.LastName) SEPARATOR '|') AS 'Patients'  
FROM YearStats ys  
JOIN Patient p ON ys.PatientAMKA = p.AMKA  
GROUP BY ys.HospitalYear, ys.TotalDays  
HAVING COUNT(ys.PatientAMKA) > 1  
ORDER BY ys.HospitalYear DESC, ys.TotalDays DESC;
```

Χρησιμοποιήσαμε CTE προκειμένου να προ-υπολογίσουμε το σύνολο των ημερών νοσηλείας ανά έτος για κάθε ασθενή, εφαρμόζοντας το φίλτρο των 15 ημερών. Στη συνέχεια, μέσω JOIN με τον πίνακα `Patient`, ομαδοποιήσαμε τα δεδομένα βάσει έτους και συνολικής διάρκειας, χρησιμοποιώντας τη συνθήκη `HAVING COUNT > 1` για τον εντοπισμό περιπτώσεων με ταυτόσημο αριθμό ημερών. Η χρήση της συνάρτησης `GROUP_CONCAT` επέτρεψε τη συγκέντρωση των ονοματεπωνύμων σε μία

ενιαία συμβολοσειρά ανά κατηγορία, ενώ η τελική ταξινόμηση (ORDER BY) παρουσιάζει τα αποτελέσματα με φθίνουσα χρονολογική και αριθμητική σειρά.

B.10

Πολλοί ασθενείς λαμβάνουν συνδυασμούς φαρμάκων. Βρείτε τα top-3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα στον ίδιο ασθενή κατά την ίδια νοσηλεία, ταξινομημένα κατά συχνότητα εμφάνισης.

SQL

```
WITH PrescribedSubstances AS (  
  SELECT pe.HospitalizationID, pe.StartDate, pe.EndDate, hs.SubID, s.Name  
  FROM PrescriptionEvent pe  
  JOIN HasSubstances hs USING (DrugID)  
  JOIN Substances s ON hs.SubID = s.ID  
)  
SELECT  
  pe1.Name AS Substance_A,  
  pe2.Name AS Substance_B,  
  COUNT(*) AS TotalOccurrences  
FROM PrescribedSubstances pe1  
JOIN PrescribedSubstances pe2  
  ON pe1.HospitalizationID = pe2.HospitalizationID  
  AND pe1.SubID < pe2.SubID  
  AND pe1.StartDate < COALESCE(pe2.EndDate, '9999-12-31')  
  AND pe2.StartDate < COALESCE(pe1.EndDate, '9999-12-31')  
GROUP BY pe1.SubID, pe2.SubID, pe1.Name, pe2.Name  
ORDER BY TotalOccurrences DESC  
LIMIT 3;
```

Για την ανίχνευση των συνηθέστερων συνδυασμών φαρμακευτικής αγωγής, υλοποιήσαμε ένα CTE που εξάγει τις δραστικές ουσίες ανά συμβάν συνταγογράφησης. Στη συνέχεια, εφαρμόσαμε *self-join* προκειμένου να εντοπίσουμε ζεύγη ουσιών που χορηγήθηκαν στον ίδιο ασθενή κατά την ίδια νοσηλεία. Ο έλεγχος της ταυτόχρονης χορήγησης επιτεύχθηκε μέσω της σύγκρισης των χρονικών διαστημάτων έναρξης και λήξης της αγωγής, χρησιμοποιώντας τη συνάρτηση COALESCE για τη διαχείριση των ανοικτών συνταγών. Τέλος, μέσω ομαδοποίησης (GROUP BY) και φθίνουσας ταξινόμησης, απομονώσαμε τα τρία επικρατέστερα ζεύγη (LIMIT 3) ουσιών.

B.11

Βρείτε όλους τους ιατρούς που έχουν εκτελέσει τουλάχιστον 5 λιγότερες επεμβάσεις από τον ιατρό με τις περισσότερες επεμβάσεις στο τρέχον έτος.

SQL

```

WITH DocVolumes AS (
    SELECT d.AMKA, d.FirstName, d.LastName, COUNT(pev.ProcEventID) AS TotalProc
    FROM DocInfo d
    LEFT JOIN ProcedureEvent pev ON pev.MainDocAMKA = d.AMKA
    AND YEAR(pev.`DateTime`) = YEAR(CURDATE())
    GROUP BY d.AMKA, d.FirstName, d.LastName
)
SELECT dv.AMKA, dv.FirstName, dv.LastName, TotalProc
FROM DocVolumes dv
WHERE TotalProc <= (
    SELECT MAX(TotalProc) - 5 FROM DocVolumes
);

```

Χρησιμοποιήσαμε ένα CTE (DocVolumes) ώστε να υπολογίσουμε το πλήθος των ιατρικών πράξεων ανά ιατρό για το τρέχον έτος, αξιοποιώντας την όψη *DocInfo* και τον πίνακα *ProcedureEvent*. Η χρήση της *LEFT JOIN* διασφαλίζει ότι θα συμπεριληφθούν και οι ιατροί με μηδενική δραστηριότητα. Στο τελικό στάδιο, εφαρμόσαμε φιλτράρισμα με *subquery* προκειμένου να απομονώσουμε εκείνους τους επαγγελματίες υγείας των οποίων το κλινικό έργο υπολείπεται κατά τουλάχιστον πέντε μονάδες από τη μέγιστη καταγεγραμμένη τιμή του έτους.

*Σημείωση: Η αρχιτεκτονική επιλογή να ενσωματωθεί το χρονικό φίλτρο *YEAR(...)=YEAR(CURDATE())* αποκλειστικά στη **φίλτρο ON του LEFT JOIN** διασφαλίζει μαθηματικά ότι οι ιατροί με μηδενική δραστηριότητα δεν θα απορριφθούν (όπως θα συνέβαινε αν τοποθετούνταν στη ρήτρα *WHERE*, μετατρέποντας τη συνένωση σε *INNER JOIN*, όπως γνωρίζουμε από την θεωρία βελτιστοποίησης ερωτημάτων)

B.12

Βρείτε τον απαιτούμενο αριθμό προσωπικού ανά τμήμα και ανά βάρδια για συγκεκριμένη εβδομάδα, με ανάλυση ανά υποκλάση προσωπικού (ιατροί ανά ειδικότητα, νοσηλευτές ανά βαθμίδα, διοικητικό προσωπικό ανά ρόλο).

```

SQL

SELECT
    hd.ShiftDate AS `Date`,
    hd.ShiftTypeName AS `ShiftType`,
    dep.Name AS `Department`,
    'Doctor' AS `StaffCategory`,
    d.Specialty AS `SubClass`,
    COUNT(hd.DoctorAMKA) AS `StaffCount`
FROM hasDoctor hd

```

```

JOIN doctor d ON hd.DoctorAMKA = d.AMKA
JOIN department dep ON hd.DepartmentID = dep.DepartmentID
WHERE hd.ShiftDate BETWEEN '2026-03-10' AND '2026-03-17'
GROUP BY hd.ShiftDate, hd.ShiftTypeName, dep.Name, d.Specialty

```

```

UNION ALL

```

```

SELECT
    hn.ShiftDate,
    hn.ShiftTypeName,
    dep.Name,
    'Nurse',
    n.Rank,
    COUNT(hn.NurseAMKA)
FROM hasNurse hn
JOIN nurse n ON hn.NurseAMKA = n.AMKA
JOIN department dep ON hn.DepartmentID = dep.DepartmentID
WHERE hn.ShiftDate BETWEEN '2026-03-10' AND '2026-03-17'
GROUP BY hn.ShiftDate, hn.ShiftTypeName, dep.Name, n.Rank

```

```

UNION ALL

```

```

SELECT
    ha.ShiftDate,
    ha.ShiftTypeName,
    dep.Name,
    'Admin',
    a.Role,
    COUNT(ha.AdminAMKA)
FROM hasAdmin ha
JOIN AdminStaff a ON ha.AdminAMKA = a.AMKA
JOIN department dep ON ha.DepartmentID = dep.DepartmentID
WHERE ha.ShiftDate BETWEEN '2026-03-10' AND '2026-03-17'
GROUP BY ha.ShiftDate, ha.ShiftTypeName, dep.Name, a.Role

```

```

ORDER BY `Date`, `Department`, `ShiftType`, `StaffCategory`;

```

Εφαρμόσαμε τη συνδυαστική μέθοδο **UNION ALL** μεταξύ τριών διακριτών ερωτημάτων συνάθροισης. Κάθε επιμέρους ερώτημα εκτελεί *JOIN* με τους αντίστοιχους πίνακες βαρδιών (hasDoctor, hasNurse, hasAdmin) και τις υποκλάσεις προσωπικού, ώστε να ανακτηθούν τα εξειδικευμένα χαρακτηριστικά (Specialty, Rank, Role). Μέσω της ομαδοποίησης (GROUP BY) ανά ημερομηνία, τμήμα και τύπο βάρδιας, επιτύχαμε τον ακριβή υπολογισμό του ανθρώπινου δυναμικού για το επιλεγμένο επταήμερο. Η τελική ταξινόμηση (ORDER BY) διασφαλίζει τη δομημένη παρουσίαση του προγράμματος υπηρεσιών.

B.13

Βρείτε για κάθε ιατρό όλη την ιεραρχία εποπτείας του, από τον άμεσο επόπτη έως τον Διευθυντή, με ένδειξη του επιπέδου σε κάθε βαθμίδα.

SQL

```
WITH RECURSIVE doctor_hierarchy AS (
  SELECT
    d.AMKA AS RootDoctorAMKA,
    CONCAT(d.FirstName, ' ', d.LastName) AS RootDoctorName,
    d.AMKA AS CurrentDoctorAMKA,
    CONCAT(d.FirstName, ' ', d.LastName) AS CurrentDoctorName,
    d.`Rank` AS CurrentDoctorRank,
    0 AS `Level`,
    d.SupervisorAMKA
  FROM DocInfo d
  UNION ALL
  SELECT
    h.RootDoctorAMKA,
    h.RootDoctorName,
    sup.AMKA AS CurrentDoctorAMKA,
    CONCAT(sup.FirstName, ' ', sup.LastName) AS CurrentDoctorName,
    sup.`Rank` AS CurrentDoctorRank,
    h.`Level` + 1 AS `Level`,
    sup.SupervisorAMKA
  FROM doctor_hierarchy h
  JOIN DocInfo sup ON h.SupervisorAMKA = sup.AMKA
)
SELECT * FROM doctor_hierarchy
ORDER BY RootDoctorAMKA, `Level`;
```

Για την υλοποίηση της αναδρομικής αναζήτησης στην ιεραρχία των ιατρών, χρησιμοποιήσαμε έναν αναδρομικό κοινό πίνακα έκφρασης (**Recursive CTE**) με την ονομασία `doctor_hierarchy`. Η διαδικασία ξεκινά με την επιλογή του ιατρού-βάση και, μέσω διαδοχικών *JOIN* με την όψη *DocInfo*, διατρέχει την αλυσίδα των εποπτών (*SupervisorAMKA*) μέχρι το ανώτατο διοικητικό επίπεδο. Σε κάθε επανάληψη, το πεδίο ``Level`` αυξάνεται κατά μία μονάδα, επιτρέποντας την ακριβή αποτύπωση της απόστασης κάθε βαθμίδας από τον αρχικό ιατρό. Το τελικό αποτέλεσμα ταξινομείται (*ORDER BY*) ανά ιατρό-ρίζα και επίπεδο ιεραρχίας, εξασφαλίζοντας τη δομημένη παρουσίαση της διοικητικής αλυσίδας.

B.14

Βρείτε ποιες κατηγορίες ICD-10 διαγνώσεων είχαν τον ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη, με τουλάχιστον 5 περιστατικά ανά έτος.

SQL

```
WITH YearlyStats AS (  
    SELECT  
        ad.ICDCode,  
        YEAR(h.AdmissionDateTime) AS YearOfEntry,  
        COUNT(h.HospitalizationID) AS CaseVolume  
    FROM Hospitalization h  
    JOIN AdmissionDiagnosis ad ON h.HospitalizationID = ad.HospitalizationID  
    GROUP BY ad.ICDCode, YEAR(h.AdmissionDateTime)  
    HAVING COUNT(h.HospitalizationID) >= 5  
)  
SELECT  
    s1.ICDCode AS 'ICD-10 Category',  
    s1.YearOfEntry AS 'First Year',  
    s2.YearOfEntry AS 'Consecutive Year',  
    s1.CaseVolume AS 'Common Admission Count'  
FROM YearlyStats s1  
JOIN YearlyStats s2 ON s1.ICDCode = s2.ICDCode  
    AND s1.CaseVolume = s2.CaseVolume  
    AND s1.YearOfEntry = s2.YearOfEntry - 1  
ORDER BY s1.YearOfEntry DESC, s1.CaseVolume DESC;
```

Χρησιμοποιώντας έναν αναδρομικό κοινό πίνακα έκφρασης (CTE), υπολογίσαμε αρχικά το πλήθος των περιστατικών ανά κωδικό ICD-10 και ανά έτος εισαγωγής, εφαρμόζοντας το φίλτρο των τουλάχιστον 5 νοσηλείων. Στη συνέχεια, μέσω της πράξης *self-join*, συσχετίσαμε τα δεδομένα για διαδοχικές χρονικές περιόδους, αναζητώντας περιπτώσεις με ταυτόσημο αριθμό εισαγωγών. Η τελική αναφορά παρουσιάζει τους κωδικούς διαγνώσεων και τα αντίστοιχα έτη, ταξινομημένα με φθίνουσα σειρά προτεραιότητας βάσει χρονολογίας και όγκου περιστατικών.

B.15

Βρείτε την κατανομή των περιστατικών triage ανά επίπεδο επείγοντος, με μέσο χρόνο αναμονής ανά επίπεδο, ποσοστό περιστατικών που οδήγησαν σε νοσηλεία και κατανομή παραπομπών ανά τμήμα.

SQL

```
WITH departmentcnt AS (  
    SELECT t.EmergencyLevel, d.`Name` AS DepartmentName, COUNT(t.TriageID) AS CasesPerDept  
    FROM TriageEvent t  
    JOIN Hospitalization h USING (HospitalizationID)  
    JOIN Department d USING (DepartmentID)  
    WHERE t.Outcome = 'Accepted'  
    GROUP BY t.EmergencyLevel, d.Name  
)
```

```

DepartmentDistribution AS (
    SELECT EmergencyLevel, GROUP_CONCAT(CONCAT(DepartmentName, ': ', CasesPerDept) SEPARATOR ' |
') AS ReferralDistribution
    FROM departmentcnt
    GROUP BY EmergencyLevel
)
SELECT t.EmergencyLevel, COUNT(t.TriageID) AS TotalTriages,
    ROUND(AVG(
        CASE
            WHEN t.Outcome IN ('Accepted', 'Discarded') THEN TIMESTAMPDIFF(MINUTE,
t.TriageDateTime, t.AssessmentDateTime)
        END
    ), 2) AS AvgWaitingTimeMinutes,
    ROUND((SUM(CASE WHEN t.Outcome = 'Accepted' THEN 1 ELSE 0 END) /
    SUM(CASE WHEN t.Outcome IN ('Accepted', 'Discarded') THEN 1 ELSE 0 END)) * 100, 2) AS
AcceptanceRate,
    COALESCE(dd.ReferralDistribution, 'No Admissions') AS DepartmentReferrals
FROM TriageEvent t
LEFT JOIN DepartmentDistribution dd USING (EmergencyLevel)
GROUP BY t.EmergencyLevel, dd.ReferralDistribution
ORDER BY t.EmergencyLevel ASC;

```

Υλοποιήσαμε μια σειρά από CTE προκειμένου να χαρτογραφήσουμε την κατανομή των παραπομπών ανά τμήμα. Μέσω της συνάρτησης *TIMESTAMPDIFF*, υπολογίσαμε τον μέσο χρόνο αναμονής από τη στιγμή της διαλογής έως την τελική ιατρική αξιολόγηση (*AssessmentDateTime*), ενώ με τη χρήση *conditional aggregation* (*CASE WHEN*) εξάγαμε το ποσοστό αποδοχής (*Acceptance Rate*) για κάθε επίπεδο επείγοντος. Η χρήση της *LEFT JOIN* με την προ-υπολογισμένη κατανομή διασφαλίζει την πληρότητα της αναφοράς, ενώ η συνάρτηση *COALESCE* διαχειρίζεται περιπτώσεις χωρίς εισαγωγές.

*Σημείωση: Ο υπολογισμός των ποσοστών αποδοχής υλοποιείται μέσω Boolean Συνάθροισης, αξιοποιώντας στοχευμένα τη συνάρτηση *SUM(CASE...)* αντί της *COUNT*, ώστε να εκτελεστεί αλγεβρική άθροιση των αληθών (1) συνθηκών και να αποφευχθεί η εσφαλμένη καταμέτρηση των ψευδών (0) τιμών.

Γ. Front End

Η διαδικτυακή εφαρμογή αλληλεπίδρασης με τη βάση δεδομένων αναπτύχθηκε σε περιβάλλον Python αξιοποιώντας το πλαίσιο (framework) Streamlit. Η εφαρμογή δεν αποτελεί απλώς ένα εργαλείο οπτικοποίησης, αλλά ακολουθεί τις αρχές της Αρχιτεκτονικής Τριών Επιπέδων (3-Tier Architecture), διαχωρίζοντας τη διεπαφή χρήστη (Presentation Layer), τη λογική διαχείρισης (Application/Business Layer) και την πρόσβαση στα δεδομένα (Data Access Layer). Η επικοινωνία με τον MySQL Server πραγματοποιείται μέσω της βιβλιοθήκης *mysql-connector-python*.

Η αρχιτεκτονική του κώδικα απομονώνει τις αρμοδιότητες σε τρία βασικά υποσυστήματα:

1. Επίπεδο Πρόσβασης Δεδομένων (Data Access Layer - db.py)

Το αρχείο αυτό περιέχει όλη τη λογική επικοινωνίας με το RDBMS, προσφέροντας ένα ασφαλές API προς την υπόλοιπη εφαρμογή:

- DatabaseConfig: Κλάση διαχείρισης των παραμέτρων σύνδεσης (host, port, user, db).
- fetch_df / execute: Συναρτήσεις γενικής χρήσης για την εκτέλεση DQL (SELECT) επιστρέφοντας τα αποτελέσματα ως δομές pandas DataFrame, και DML (INSERT/UPDATE/DELETE) εντολών αντίστοιχα.
- execute transaction: Υλοποιεί ασφαλή εκτέλεση πολλαπλών εντολών εντός ενιαίου Transaction (εξασφαλίζοντας την ιδιότητα ACID).
- call returning df: Διαχειρίζεται την κλήση Αποθηκευμένων Διαδικασιών (Stored Procedures) που επιστρέφουν παραμέτρους (OUT) ή result sets.
- Εξειδικευμένες μέθοδοι (calculate_bill, admit_new_patient, generate_shifts) που αντιστοιχούν σε βασικά επιχειρησιακά σενάρια (use cases).

2. Διαχείριση Ερωτημάτων (Query Repository - queries.py)

Το επίπεδο αυτό λειτουργεί ως “αποθήκη” των πολύπλοκων αναλυτικών ερωτημάτων (Q1 έως Q15). Παρέχει μηχανισμούς δυναμικής παραμετροποίησης (parameterized queries) για τα ερωτήματα που απαιτούν είσοδο από τον χρήστη (όπως τα Q2, Q4, Q6, Q8, Q12), προστατεύοντας παράλληλα την εφαρμογή από επιθέσεις τύπου SQL Injection.

3. Επίπεδο Παρουσίασης και Δρομολόγησης (Presentation Layer - app.py)

Αποτελεί τον κεντρικό ελεγκτή (controller) της εφαρμογής, διαχειρίζεται το routing μέσω του πλευρικού μενού (sidebar) και συγκροτείται από τις ακόλουθες υποσελίδες/ενότητες:

Dashboard: Παρουσιάζει τη γενική κατάσταση του νοσοκομείου. Εκτελεί aggregate queries για την προβολή KPIs (πλήθος ιατρών, ασθενών, νοσηλειών), ενσωματώνει απευθείας τα SQL Views (DepartmentInfo, ShiftsAlerts, PatientQueue) και παράγει γραφήματα για την ουρά διαλογής και το ιστορικό των Triage Outcomes.

Queries Q1-Q15: Εξειδικευμένο περιβάλλον εκτέλεσης των 15 παραδοτέων ερωτημάτων. Προβάλλει τον raw SQL κώδικα σε text area για επίδειξη και εκτελεί τα ερωτήματα σε read-only mode, διασφαλίζοντας την ακεραιότητα της βάσης από τυχαίες παρεμβάσεις του χρήστη.

Triage Workflow: Υλοποιεί την πλήρη ροή των ΤΕΠ. Απεικονίζει τα εκκρεμή (pending) περιστατικά, εξάγει το επόμενο περιστατικό βάσει προτεραιότητας μέσω της ρουτίνας FetchNext, και επιτρέπει την εισαγωγή ή απόρριψη του ασθενούς καλώντας τις διαδικασίες ProcessTriageAdmission και DiscardTriagePatient αντίστοιχα.

Hospitalizations: Περιβάλλον διαχείρισης νοσηλειών. Καλύπτει την προβολή του ενεργού ιστορικού, την εισαγωγή νέων ασθενών, τη διεκπεραίωση του εξιτηρίου (DischargePatient) και την αυτόματη κοστολόγηση βάσει KEN μέσω της ρουτίνας CalculateHospitalizationBill.

Staff and Shifts: Διαχειρίζεται τον προγραμματισμό του ανθρώπινου δυναμικού. Επιτρέπει την ανάθεση βαρδιών στο προσωπικό. Σε αυτό το σημείο, το UI λειτουργεί ως λεπτός πελάτης (thin client): δεν αναπαράγει την επιχειρησιακή λογική, αλλά αναθέτει τον έλεγχο των κανόνων (π.χ. μέγιστες βάρδιες, δωρη ανάπαυση) αυστηρά στα Triggers της βάσης, λαμβάνοντας και προβάλλοντας τα SQL Exceptions σε περίπτωση παραβίασης των κανόνων.

Prescription Safety: Φόρμα ασφαλούς συνταγογράφησης. Πριν την υποβολή της συνταγής, το UI προβάλλει προειδοποιητικά τις γνωστές αλλεργίες του ασθενούς. Η τελική έγκριση ωστόσο και το μπλοκάρισμα της συνταγής σε περίπτωση αλλεργικής αντίδρασης, εγγυάται αδιαπέραστα από το αντίστοιχο trigger του DBMS (trg_patient_allergy_check).

Images and Catalogs: Διεπαφή προβολής των στατικών καταλόγων (Cost, DrugType) και των συνδεδεμένων πολυμέσων (Entity: Image) των εγκαταστάσεων και του προσωπικού.

Δ. Χρήση AI/LLMs:

Κατά την εκπόνηση της παρούσας εργασίας χρησιμοποιήθηκαν εργαλεία Τεχνητής Νοημοσύνης και LLM (OpenAI ChatGPT, και Google NotebookLM) με αυστηρά υποστηρικτικό και εποικοδομητικό ρόλο.

Πιο συγκεκριμένα, η συμβολή των εργαλείων εντοπίζεται στους εξής τομείς:

1. **Ακαδημαϊκή Τεκμηρίωση και Συγγραφή της Αναφοράς** : Ο βασικός σχεδιασμός (ER και Relational Model), οι αρχιτεκτονικές αποφάσεις και η συγγραφή των SQL queries ήταν αποκλειστικά αποτέλεσμα δικής μας μελέτης και κριτικής σκέψης. Ωστόσο, τα εργαλεία LLM, συγκεκριμένα το NotebookLM, χρησιμοποιήθηκαν για τον γλωσσικό έλεγχο και την αναβάθμιση του τεχνικού λεξιλογίου, βοηθώντας μας να μεταφράσουμε με ακρίβεια τις τεχνικές μας αποφάσεις (π.χ. επιλογές ευρετηρίων, κανονικοποίηση 3NF) σε ορθό ακαδημαϊκό λόγο.
2. **Παραγωγή Συνθετικών Δεδομένων & Προεπεξεργασία**: Για την κάλυψη των αυστηρών απαιτήσεων ελάχιστου όγκου δεδομένων της εκφώνησης, τα εργαλεία LLM, συγκεκριμένα το ChatGPT, χρησιμοποιήθηκαν για την δημιουργία synthetic data , παράγοντας το σύνολο των τελικών συνθετικών αρχείων CSV (περιλαμβάνοντας χιλιάδες εγγραφές για ιατρούς, ασθενείς, νοσηλείες, βάρδιες κ.λπ.). Παράλληλα, τα συστήματα αυτά αξιοποιήθηκαν για την προεπεξεργασία των επίσημων, ακατέργαστων αρχείων της εκφώνησης (μορφής .xls, .docx και .csv, όπως οι κατάλογοι ICD-10, KEN και φαρμάκων του EMA). Μέσω του AI, τα δεδομένα αυτά καθαρίστηκαν, μετασχηματίστηκαν και προσαρμόστηκαν αυστηρά στις απαιτήσεις του δικού μας σχεσιακού σχήματος, ώστε να είναι έτοιμα να φορτωθούν αυτούσια στους πίνακες της βάσης μέσω του αρχείου **load.sql**.
3. **Ανάπτυξη Διεπαφής Χρήστη (UI)**: Στην υλοποίηση της διαδικτυακής εφαρμογής, τα εργαλεία LLM, συγκεκριμένα το ChatGPT, χρησιμοποιήθηκαν αρχικά για την παραγωγή του βασικού, κώδικα της διεπαφής . Ωστόσο, αφιερώσαμε σημαντικό χρόνο στη επεξεργασία και τη αναδιαμόρφωση της διεπαφής, μελετώντας τις δυνατότητές της προκειμένου να την προσαρμόσουμε στις δικές μας εξειδικευμένες απαιτήσεις. Εστίασαμε στον αυστηρό έλεγχο και στην ενσωμάτωση του front-end με το περίπλοκο back-end (π.χ. κλήση των Stored Procedures, άντληση δεδομένων από τα Views). Αυτή η στρατηγική προσέγγιση μας επέτρεψε να εξοικονομήσουμε πολύτιμο χρόνο από τον σχεδιασμό του ui και να επικεντρωθούμε στην ουσία της εργασίας, δηλαδή την ανάπτυξη της βάσης μας και της ασφαλούς σύνδεσης της με το front-end.

4. **Debugging:** Τα συστήματα (όπως το NotebookLM, τροφοδοτούμενο με το υλικό του μαθήματος) χρησιμοποιήθηκαν ως εργαλεία εκμάθησης για την αποσαφήνιση προχωρημένων εννοιών και λειτουργιών της MySQL (π.χ. την αυτόματη δημιουργία index για unique & foreign keys, την απαγόρευση χρήσης foreign key σε check constraints, την απαγόρευση χρήσης των μη ντετερμινιστικών συναρτήσεων σε check constraints όπως η *curdate()*, την δομή και ασφαλή χρήση των transactions με exit handlers) και για τον γρήγορο εντοπισμό συντακτικών σφαλμάτων στα DDL scripts των Triggers & Stored Procedures.